

UDESC

Universidade do Estado de Santa Catarina

é sempre o XOR

Eduardo Schwarz Moreira, Enzo Rodrigues, João Oliveira

2026-02-28

Contest (1)

```
.vimrc
7 lines

set nu ai si cindent et ts=4 sw=4 so=10 nosm undofile

inoremap {} {}<left><return><up><end><return>
" remap de chaves

au BufReadPost * if line("'\"") > 0 && line("'\"") <= line("$")
    | exe "normal! g'\"" | endif
" volta pro lugar onde estava quando saiu do arquivo

run.sh
2 lines

#!/bin/bash
g++ -std=c++20 -DBRUTE -O2 -Wall -Wextra -Wconversion -Wfatal-
errors -fsanitize=address,undefined $1 && ./a.out

hash.sh
3 lines

# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]' | md5sum |cut -c-6

Mint.cpp
d41d8c, 52 lines

template <auto MOD, typename T = decltype(MOD)>
struct Mint {
    using U = long long;
    // se o modulo for long long, usar U = __int128
    using m = Mint<MOD, T>;
    T v;
    Mint(T val = 0) : v(val) {
        assert(sizeof(T) * 2 <= sizeof(U));
        if (v < -MOD || v >= 2 * MOD) v %= MOD;
        if (v < 0) v += MOD;
        if (v >= MOD) v -= MOD;
    }
    Mint(U val) : v(T(val % MOD)) {
        assert(sizeof(T) * 2 <= sizeof(U));
        if (v < 0) v += MOD;
    }
    bool operator==(m o) const { return v == o.v; }
    bool operator<(m o) const { return v < o.v; }
    bool operator!=(m o) const { return v != o.v; }
    m pwr(m b, U e) const {
        m res = 1;
        while (e > 0) {
            if (e & 1) res *= b;
            b *= b, e /= 2;
        }
        return res;
    }
    m &operator+=(m o) {
        v -= MOD - o.v;
        if (v < 0) v += MOD;
        return *this;
    }
    m &operator-=(m o) {
        v -= o.v;
        if (v < 0) v += MOD;
        return *this;
    }
    m &operator*=(m o) {
        v = (T)((U)v * o.v % MOD);
        return *this;
    }
    m &operator/=(m o) { return *this *= o.pwr(o, MOD - 2); }
```

.vimrc run hash Mint

```
m &operator^=(U e) { return *this = pwr(*this, e); }
friend m operator-(m a, m b) { return a -= b; }
friend m operator+(m a, m b) { return a += b; }
friend m operator*(m a, m b) { return a *= b; }
friend m operator/(m a, m b) { return a /= b; }
friend m operator^(m a, U e) { return a.pwr(a, e); }

m operator-( ) { return m(this->v ? MOD - this->v : 0); }
m inv() const { return pwr(*this, MOD - 2); } // MOD must
    be prime

};
```

Mathematics (2)

2.1 Trigonometry

sin(v + w) = sin v cos w + cos v sin w

cos(v + w) = cos v cos w − sin v sin w

tan(v + w) = tan v + tan w / (1 − tan v tan w)

sin v + sin w = 2 sin (v + w) / 2 cos (v − w) / 2

cos v + cos w = 2 cos (v + w) / 2 cos (v − w) / 2

(V + W) tan(v − w) / 2 = (V − W) tan(v + w) / 2

where V, W are lengths of sides opposite angles v, w.

a cos x + b sin x = r cos(x − ϕ)

a sin x + b cos x = r sin(x + ϕ)

where r = √a² + b², ϕ = atan2(b, a).

2.2 Geometry

2.2.1 Triangles

Side lengths: a, b, c

Semiperimeter: p = (a + b + c) / 2

Area: A = √p(p − a)(p − b)(p − c)

Circumradius: R = abc / 4A

Inradius: r = A / p

Length of median (divides triangle into two equal-area triangles):

m_a = 1/2 √2b² + 2c² − a²

Length of bisector (divides angles in two):

s_a = √bc [1 − (a / (b + c))²]

Law of sines: sin α / a = sin β / b = sin γ / c = 1 / 2R

Law of cosines: a² = b² + c² − 2bc cos α

Law of tangents: (a + b) / (a − b) = tan (α + β) / 2 / tan (α − β) / 2

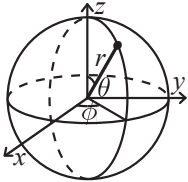
2.2.2 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux F = b² + d² − a² − c²:

4A = 2ef · sin θ = F tan θ = √4e²f² − F²

For cyclic quadrilaterals the sum of opposite angles is 180°, ef = ac + bd, and A = √(p − a)(p − b)(p − c)(p − d).

2.2.3 Spherical coordinates



x = r sin θ cos ϕ r = √x² + y² + z²

y = r sin θ sin ϕ θ = acos(z / √x² + y² + z²)

z = r cos θ ϕ = atan2(y, x)

2.3 Derivatives/Integrals

d/dx arcsin x = 1 / √1 − x² d/dx arccos x = −1 / √1 − x²

d/dx tan x = 1 + tan² x d/dx arctan x = 1 / (1 + x²)

∫ tan ax = −ln |cos ax| / a ∫ x sin ax = (sin ax − ax cos ax) / a²

∫ e⁻ˣ² = √π / 2 erf(x) ∫ xeᵃˣ dx = eᵃˣ / a² (ax − 1)

Integration by parts:

∫ₐᵇ f(x)g(x)dx = [F(x)g(x)]ₐᵇ − ∫ₐᵇ F(x)g'(x)dx

2.4 Sums

cᵃ + cᵃ⁺¹ + ⋯ + cᵇ = (cᵇ⁺¹ − cᵃ) / (c − 1), c ≠ 1

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

$$1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.5 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots, (-1 < x \leq 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \dots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots, (-\infty < x < \infty)$$

2.6 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x . It will then have an expected value (mean)

$\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y ,

$$V(aX + bY) = a^2 V(X) + b^2 V(Y).$$

2.7 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \dots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

π is a stationary distribution if $\pi = \pi \mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i . π_j / π_i is the expected number of visits in state j between two visits in state i .

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i 's degree.

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k \rightarrow \infty} \mathbf{P}^k = \mathbf{1P}$.

A Markov chain is an A-chain if the states can be partitioned into two sets $\mathbf{A}$ and $\mathbf{G}$, such that all states in $\mathbf{A}$ are absorbing ($p_{ii} = 1$), and all states in $\mathbf{G}$ leads to an absorbing state in $\mathbf{A}$. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j , is $a_{ij} = p_{ij} + \sum_{k \in \mathbf{G}} a_{ik} p_{kj}$. The expected time until absorption, when the initial state is i , is $t_i = 1 + \sum_{k \in \mathbf{G}} p_{ki} t_k$.

Data structures (3)

OrderedSet.cpp

Description: Set with operations to find the element by index - find_by_order(i) and to find the index of an element

Usage: ordered_set<int> seta;

Time: All operations in $\mathcal{O}(\log N)$.

<ext/pb_ds/assoc.container.hpp>, <ext/pb_ds/tree_policy.hpp> d41d8c, 8 lines

using namespace __gnu_pbds;

```
template <typename T>
using ordered_set =
    tree<T, null_type, less<T>, rb_tree_tag,
        tree_order_statistics_node_update>;
```

```
template <typename T, typename U>
using ordered_map = tree<T, U, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
```

SegmentTreeBeats.cpp

Description: Updates of max, min and sum in range with queries of sum in range

Usage: "seg" is already declared globally, just call build

Time: If sum update is not used: $\mathcal{O}(\log N)$, otherwise $\mathcal{O}(\log^2 N)$

```
const ll INF = 1e18;
struct node {
    ll mi, smi, mx, smx, sum, lazy;
    int fmi, fmx;
    node() {
        mi = smi = INF;
        mx = smx = -INF;
        fmi = 0, fmx = 0, sum = 0, lazy = 0;
    }
    node(ll val) {
        mi = mx = sum = val;
        smi = INF, smx = -INF;
        fmx = fmi = 1;
        lazy = 0;
    }
};
```

```
};

node operator+(node a, node b) {
    node ret;
    ret.sum = a.sum + b.sum;
    if (a.mi == b.mi) {
        ret.mi = a.mi;
        ret.fmi = a.fmi + b.fmi;
        ret.smi = min(a.smi, b.smi);
    } else if (a.mi < b.mi) {
        ret.mi = a.mi;
        ret.fmi = a.fmi;
        ret.smi = min(a.smi, b.mi);
    } else {
        ret.mi = b.mi;
        ret.fmi = b.fmi;
        ret.smi = min(b.smi, a.mi);
    }
    if (a.mx == b.mx) {
        ret.mx = a.mx;
        ret.fmx = a.fmx + b.fmx;
        ret.smx = max(a.smx, b.smx);
    } else if (a.mx > b.mx) {
        ret.mx = a.mx;
        ret.fmx = a.fmx;
        ret.smx = max(b.mx, a.smx);
    } else {
        ret.fmx = b.fmx;
        ret.mx = b.mx;
        ret.smx = max(a.mx, b.smx);
    }
    return ret;
}

struct SegBeats {
    vector<node> t;
    int n;
    void build(int _n) { // pra construir com tamanho, mas
        vazia
        n = _n;
        t.assign(n * 4, node());
    }
    void build(const vector<ll> &v) { // pra construir com
        vector
        n = (int)v.size();
        t.assign(n * 4, node());
        build(1, 0, n - 1, v);
    }
    void build(ll *bg, ll *en) { // pra construir com array de
        C
        build(vector<ll>(bg, en));
    }
    inline int lc(int p) { return 2 * p; }
    inline int rc(int p) { return 2 * p + 1; }
    node build(int p, int l, int r, const vector<ll> &a) {
        if (l == r) return t[p] = node(a[l]);
        int mid = (l + r) >> 1;
        return t[p] = build(lc(p), l, mid, a) + build(rc(p),
            mid + 1, r, a);
    }
    void pushsum(int p, int l, int r, ll x) {
        t[p].sum += (r - l + 1) * x;
        t[p].mi += x;
        t[p].mx += x;
        t[p].lazy += x;
        if (t[p].smi != INF) t[p].smi += x;
        if (t[p].smx != -INF) t[p].smx += x;
    }
};
```

```

void pushmax(int p, ll x) {
    if (x <= t[p].mi) return;
    t[p].sum += t[p].fmi * (x - t[p].mi);
    if (t[p].mx == t[p].mi) t[p].mx = x;
    if (t[p].smx == t[p].mi) t[p].smx = x;
    t[p].mi = x;
}

void pushmin(int p, ll x) {
    if (x >= t[p].mx) return;
    t[p].sum += t[p].fmx * (x - t[p].mx);
    if (t[p].mi == t[p].mx) t[p].mi = x;
    if (t[p].smi == t[p].mx) t[p].smi = x;
    t[p].mx = x;
}

void pushdown(int p, int l, int r) {
    if (l == r) return;
    int mid = (l + r) >> 1;
    pushsum(lc(p), l, mid, t[p].lazy);
    pushsum(rc(p), mid + 1, r, t[p].lazy);
    t[p].lazy = 0;

    pushmax(lc(p), t[p].mi);
    pushmax(rc(p), t[p].mi);

    pushmin(lc(p), t[p].mx);
    pushmin(rc(p), t[p].mx);
}

node updatemin(int p, int l, int r, int L, int R, ll x) {
    if (l > R || r < L || x >= t[p].mx) return t[p];
    if (l >= L && r <= R && x > t[p].smx) {
        pushmin(p, x);
        return t[p];
    }
    pushdown(p, l, r);
    int mid = (l + r) >> 1;
    t[p] = updatemin(lc(p), l, mid, L, R, x) + updatemin(rc(p), mid + 1, r, L, R, x);
    return t[p];
}

node updatemax(int p, int l, int r, int L, int R, ll x) {
    if (l > R || r < L || x <= t[p].mi) return t[p];
    if (l >= L && r <= R && x < t[p].smi) {
        pushmax(p, x);
        return t[p];
    }
    pushdown(p, l, r);
    int mid = (l + r) >> 1;
    t[p] = updatemax(lc(p), l, mid, L, R, x) + updatemax(rc(p), mid + 1, r, L, R, x);
    return t[p];
}

node updatesum(int p, int l, int r, int L, int R, ll x) {
    if (l > R || r < L) return t[p];
    if (l >= L && r <= R) {
        pushsum(p, l, r, x);
        return t[p];
    }
    pushdown(p, l, r);
    int mid = (l + r) >> 1;
    return t[p] = updatesum(lc(p), l, mid, L, R, x) + updatesum(rc(p), mid + 1, r, L, R, x);
}

node query(int p, int l, int r, int L, int R) {
    if (l > R || r < L) return node();
    if (l >= L && r <= R) return t[p];
    pushdown(p, l, r);
    int mid = (l + r) >> 1;
    return query(lc(p), l, mid, L, R) + query(rc(p), mid + 1, r, L, R);
}

```

```

}

ll query(int l, int r) { return query(1, 0, n - 1, l, r).sum; }

void updatemax(int l, int r, ll x) { updatemax(1, 0, n - 1, l, r, x); }

void updatemin(int l, int r, ll x) { updatemin(1, 0, n - 1, l, r, x); }

void updatesum(int l, int r, ll x) { updatesum(1, 0, n - 1, l, r, x); }

} seg;

```

SegmentTree2D.cpp

Description: Segment Tree with point update and rectangular range query

Usage: "seg" declared globally

Time: $\mathcal{O}(\log^2 N)$

d41d8c, 47 lines

```

struct SegTree2D {
    ll merge(ll a, ll b) { return a + b; }
    ll neutral = 0;
    int n, m;
    vector<vector<ll>> t;
    void build(int _n, int _m) {
        n = _n, m = _m;
        t.assign(2 * n, vector<ll>(2 * m, neutral));
        for (int i = 2 * n - 1; i >= n; i--)
            for (int j = m - 1; j > 0; j--)
                t[i][j] = merge(t[i][j << 1], t[i][j << 1 | 1]);
        for (int i = n - 1; i > 0; i--)
            for (int j = 2 * m - 1; j > 0; j--)
                t[i][j] = merge(t[i << 1][j], t[i << 1 | 1][j]);
    }

    ll inner_query(int idx, int l, int r) {
        ll res = neutral;
        for (l += m, r += m + 1; l < r; l >>= 1, r >>= 1) {
            if (l & 1) res = merge(res, t[idx][l++]);
            if (r & 1) res = merge(res, t[idx][--r]);
        }
        return res;
    }

    // query do ponto (a, b) ate o ponto (c, d), retorna neutro se a > c ou b > d
    ll query(int a, int b, int c, int d) {
        ll res = neutral;
        for (a += n, c += n + 1; a < c; a >>= 1, c >>= 1) {
            if (a & 1) res = merge(res, inner_query(a++, b, d));
            if (c & 1) res = merge(res, inner_query(--c, b, d));
        }
        return res;
    }

    void inner_update(int idx, int i, ll x) {
        auto &c = t[idx];
        i += m;
        c[i] = x;
        for (i >>= 1; i > 0; i >>= 1) c[i] = merge(c[i << 1], c[i << 1 | 1]);
    }

    void update(int i, int j, ll x) {
        i += n;
        inner_update(i, j, x);
        for (i >>= 1; i > 0; i >>= 1) {
            ll val = merge(t[i << 1][j + m], t[i << 1 | 1][j + m]);
            inner_update(i, j, val);
        }
    }
}

```

```

} seg;

```

ImplicitTreap.cpp

Description: Does everything

Time: Expected $\mathcal{O}(\log N)$

d41d8c, 85 lines

```

mt19937 rng((int) chrono::steady_clock::now().time_since_epoch().count());

template<typename T> struct treap {
    struct node {
        node *l, *r;
        int p, sz;
        T val, sub, lazy;
        bool rev;
        node(T v) : l(NULL), r(NULL), p(rng()), sz(1), val(v), sub(v), lazy(0), rev(0) {}
        void prop() {
            if (lazy) {
                val += lazy, sub += lazy*sz;
                if (l) l->lazy += lazy;
                if (r) r->lazy += lazy;
            }
            if (rev) {
                swap(l, r);
                if (l) l->rev ^= 1;
                if (r) r->rev ^= 1;
            }
            lazy = 0, rev = 0;
        }
        void update() {
            sz = 1, sub = val;
            if (l) l->prop(), sz += l->sz, sub += l->sub;
            if (r) r->prop(), sz += r->sz, sub += r->sub;
        }
    };

    node* root;

    treap() { root = NULL; }
    treap(const treap& t) {
        throw logic_error("Nao copiar a treap!");
    }
    ~treap() {
        vector<node*> q = {root};
        while (q.size()) {
            node* x = q.back(); q.pop_back();
            if (!x) continue;
            q.push_back(x->l), q.push_back(x->r);
            delete x;
        }
    }

    int size(node* x) { return x ? x->sz : 0; }
    int size() { return size(root); }
    void join(node* l, node* r, node*& i) { // assume que l < r
        if (!l or !r) return void(i = l ? l : r);
        l->prop(), r->prop();
        if (l->p > r->p) join(l->r, r, l->r), i = l;
        else join(l, r->l, r->l), i = r;
        i->update();
    }

    void split(node* i, node*& l, node*& r, int v, int key = 0) {
        if (!i) return void(r = l = NULL);
        i->prop();
        if (key + size(i->l) < v) split(i->r, i->r, r, v, key+size(i->l)+1), l = i;
        else split(i->l, l, i->l, v, key), r = i;
        i->update();
    }
}

```

```

}
void push_back(T v) {
    node* i = new node(v);
    join(root, i, root);
}
T query(int l, int r) {
    node *L, *M, *R;
    split(root, M, R, r+1), split(M, L, M, l);
    T ans = M->sub;
    join(L, M, M), join(M, R, root);
    return ans;
}
void update(int l, int r, T s) {
    node *L, *M, *R;
    split(root, M, R, r+1), split(M, L, M, l);
    M->lazy += s;
    join(L, M, M), join(M, R, root);
}
void reverse(int l, int r) {
    node *L, *M, *R;
    split(root, M, R, r+1), split(M, L, M, l);
    M->rev ^= 1;
    join(L, M, M), join(M, R, root);
}
};
```

LichaoLazy.cpp
Description: Sendo $N = MA - MI$: insert((a, b)) minimiza tudo com $ax + b$ $O(\log N)$ insert((a, b), l, r) minimiza com $ax + b$ no range $[l, r]$ $O(\log^2 N)$ shift((a, b)) soma $ax + b$ em tudo $O(1)$ shift((a, b), l, r) soma $ax + b$ no range $[l, r]$ $O(\log^2 N)$ query(x) retorna o valor da posicao x $O(\log N)$ No inicio eh tudo LINF, se inserir (0, 0) fica tudo 0
Time: $\mathcal{O}(n \log N)$ de memoria, $\mathcal{O}(n)$ de memoria se nao usar as operacoes de range

d41d8c, 77 lines

```

template<int MI = int(-1e9), int MA = int(1e9)> struct lichao {
    struct line {
        ll a, b;
        ll la, lb; // lazy
        array<int, 2> ch;
        line(ll a_ = 0, ll b_ = LINF) :
            a(a_), b(b_), la(0), lb(0), ch({-1, -1}) {}
        ll operator ()(ll x) { return a*x + b; }
    };
    vector<line> ln;

    int ch(int p, int d) {
        if (ln[p].ch[d] == -1) {
            ln[p].ch[d] = ln.size();
            ln.emplace_back();
        }
        return ln[p].ch[d];
    }
    lichao() { ln.emplace_back(); }

    void prop(int p, int l, int r) {
        if (ln[p].la == 0 and ln[p].lb == 0) return;
        ln[p].a += ln[p].la, ln[p].b += ln[p].lb;
        if (l != r) {
            int pl = ch(p, 0), pr = ch(p, 1);
            ln[pl].la += ln[p].la, ln[pl].lb += ln[p].lb;
            ln[pr].la += ln[p].la, ln[pr].lb += ln[p].lb;
        }
        ln[p].la = ln[p].lb = 0;
    }

    ll query(int x, int p=0, int l=MI, int r=MA) {
        prop(p, l, r);
        ll ret = ln[p](x);
    }
};
```

```

    if (ln[p].ch[0] == -1 and ln[p].ch[1] == -1) return ret;
    int m = l + (r-l)/2;
    if (x <= m) return min(ret, query(x, ch(p, 0), l, m));
    return min(ret, query(x, ch(p, 1), m+1, r));
}

void push(line s, int p, int l, int r) {
    prop(p, l, r);
    int m = l + (r-l)/2;
    bool L = s(l) < ln[p](l);
    bool M = s(m) < ln[p](m);
    bool R = s(r) < ln[p](r);
    if (M) swap(ln[p].a, s.a), swap(ln[p].b, s.b);
    if (s.b == LINF) return;
    if (s.b == LINF) return;
    if (L != M) push(s, ch(p, 0), l, m);
    else if (R != M) push(s, ch(p, 1), m+1, r);
}

void insert(line s, int a=MI, int b=MA, int p=0, int l=MI,
            int r=MA) {
    prop(p, l, r);
    if (a <= l and r <= b) return push(s, p, l, r);
    if (b < l or r < a) return;
    int m = l + (r-l)/2;
    insert(s, a, b, ch(p, 0), l, m);
    insert(s, a, b, ch(p, 1), m+1, r);
}

void shift(line s, int a=MI, int b=MA, int p=0, int l=MI, int
            r=MA) {
    prop(p, l, r);
    int m = l + (r-l)/2;
    if (a <= l and r <= b) {
        ln[p].la += s.a, ln[p].lb += s.b;
        return;
    }
    if (b < l or r < a) return;
    if (ln[p].b != LINF) {
        push(ln[p], ch(p, 0), l, m);
        push(ln[p], ch(p, 1), m+1, r);
        ln[p].a = 0, ln[p].b = LINF;
    }
    shift(s, a, b, ch(p, 0), l, m);
    shift(s, a, b, ch(p, 1), m+1, r);
}
};
```

OperationDeque.cpp
Description: Deque that supports operations (min/max/gcd/dp etc) in $O(1)$
Usage: op_deque<type, Operation> dq;
Time: amortized $\mathcal{O}(1)$ per operation

d41d8c, 33 lines

```

template <typename T, auto OP>
struct op_deque {
    op_stack<T, OP> in, out;
    void push_back(T x) { in.push(x); }
    void push_front(T x) { out.push(x); }
    void work() {
        op_stack<T, OP> to;
        bool sw = false;
        if (in.empty()) sw = true, swap(in, out);
        int m = in.size();
        for (int i = 0; i < m / 2; i++) to.push(in.top()), in.
            pop();
        while (in.size()) out.push(in.top()), in.pop();
        while (to.size()) in.push(to.top()), to.pop();
        if (sw) swap(in, out);
    }
    T pop_front() {
```

```

        if (out.empty()) work();
        T ret = out.top();
        out.pop();
        return ret;
    }
    T pop_back() {
        if (in.empty()) work();
        T ret = in.top();
        in.pop();
        return ret;
    }
    T get() {
        if (in.empty()) return out.get();
        if (out.empty()) return in.get();
        return OP(in.get(), out.get());
    }
};
```

Numerical (4)

4.1 Factorization

PollardRhoMillerRabin.cpp

Description: Pollard Rho and Miller Rabin

d41d8c, 87 lines

```

namespace MillerRabin {
    inline ll mul_mod(ll a, ll b, ll m) { return (ll)((__int128
        )a * b % m); }
    inline ll power(ll b, ll e, ll m) {
        ll r = 1;
        b = b % m;
        while (e > 0) {
            if (e & 1) r = mul_mod(r, b, m);
            b = mul_mod(b, b, m), e >>= 1;
        }
        return r;
    }
    inline bool composite(ll n, ll a, ll d, ll s) {
        ll x = power(a, d, n);
        if (x == 1 || x == n - 1 || a % n == 0) return false;
        for (int r = 1; r < s; r++) {
            x = mul_mod(x, x, n);
            if (x == n - 1) return false;
        }
        return true;
    }
}

// com esses "primos", o teste funciona garantido para n <=
// 2^64
int primes[] = {2, 325, 9375, 28178, 450775, 9780504,
    1795265022};

// funciona para n <= 3*10^24 com os primos ate 41, mas tem
// que cuidar com overflow
// int primes[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
// 37, 41};

bool prime(ll n) {
    if (n <= 2 || (n % 2 == 0)) return n == 2;
    ll d = n - 1, r = 0;
    while (d % 2 == 0) d /= 2, r++;
    for (int a : primes)
        if (composite(n, a, d, r)) return false;
    return true;
}

namespace PollardRho {
```

```
mt19937 rng((uint32_t)chrono::steady_clock::now().
    time_since_epoch().count());
const ll P = 1e6 + 1;
ll seq[P];
inline ll add_mod(ll x, ll y, ll m) { return (x += y) < m ?
    x : x - m; }
inline ll mul_mod(ll a, ll b, ll m) { return (ll)((__int128
    )a * b % m); }
ll rho(ll n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    ll x0 = rng() % n, c = rng() % n;
    while (1) {
        ll x = x0++, y = x, u = 1, v, t = 0;
        ll *px = seq, *py = seq;
        while (1) {
            *py++ = y = add_mod(mul_mod(y, y, n), c, n);
            *py++ = y = add_mod(mul_mod(y, y, n), c, n);
            if ((x = *px++) == y) break;
            v = u;
            u = mul_mod(u, abs(y - x), n);
            if (!u) return gcd(v, n);
            if (++t == 32) {
                t = 0;
                if ((u = gcd(u, n)) > 1 && u < n) return u;
            }
        }
        if (t && (u = gcd(u, n)) > 1 && u < n) return u;
    }
}

vector<ll> factorize(ll x) {
    vector<ll> f;
    if (x == 1) return f;
    function<void(ll)> dfs = [&](ll x) {
        if (x == 1) return;
        if (x < Sieve::P) {
            auto fs = Sieve::factorize(x);
            f.insert(f.end(), fs.begin(), fs.end());
        } else if (MillerRabin::prime(x)) {
            f.push_back(x);
        } else {
            ll d = PollardRho::rho(x);
            dfs(d);
            dfs(x / d);
        }
    };
    dfs(x);
    sort(f.begin(), f.end());
    return f;
}
```

4.2 Polynomials and recurrences

```
TaylorShift.cpp
template <auto MOD, typename T = Mint<MOD>>
vector<T> shift(vector<T> a, int k) {
    int n = (int)a.size();
    vector<T> fat(n, 1), ifat(n, shifting(n));
    for (int i = 1; i < n; i++) fat[i] = fat[i - 1] * i;
    ifat[n - 1] = T(1) / fat[n - 1];
    for (int i = n - 1; i > 0; i--) ifat[i - 1] = ifat[i] * i;
    for (int i = 0; i < n; i++) a[i] *= fat[i];
    T pk = 1;
    for (int i = 0; i < n; i++) {
        shifting[n - i - 1] = pk * ifat[i];
        pk *= k;
    }
}
```

```
auto ans = multiply<MOD>(a, shifting);
ans.erase(ans.begin(), ans.begin() + n - 1);
for (int i = 0; i < n; i++) ans[i] *= ifat[i];
return ans;
}

EvaluateInterpolation.cpp
Description: Dado 'n' pontos (i, y[i]), i ∈ [0, n), avalia o polinomio de grau
n-1 que passa por esses pontos em 'x' Tudo modular, precisa do mint
Time: O(N)
mint evaluate_interpolation(int x, vector<mint> y) {
    int n = y.size();

    vector<mint> sulf(n+1, 1), fat(n, 1), ifat(n);
    for (int i = n-1; i >= 0; i--) sulf[i] = sulf[i+1] * (x - i);
    for (int i = 1; i < n; i++) fat[i] = fat[i-1] * i;
    ifat[n-1] = 1/fat[n-1];
    for (int i = n-2; i >= 0; i--) ifat[i] = ifat[i+1] * (i + 1);

    mint pref = 1, ans = 0;
    for (int i = 0; i < n; pref *= (x - i++)) {
        mint num = pref * sulf[i+1];

        mint den = ifat[i] * ifat[n-1 - i];
        if ((n-1 - i)%2) den *= -1;

        ans += y[i] * num * den;
    }
    return ans;
}
```

```
polinomio.cpp
Description: struct bizarramente overkill de polinomio mas faz tudo so vai
demorar pra copiar eh a vida
struct poly {
    vector<mint> a;
    inline void normalize() {
        while ((int)a.size() && a.back() == 0) a.pop_back();
    }
    template <class... Args>
    poly(Args... args) : a(args...) {}
    poly(const initializer_list<mint> &x) : a(x.begin(), x.end
        ()) {}
    int size() const { return (int)a.size(); }
    inline mint coef(const int i) const {
        return (i < a.size() && i >= 0) ? a[i] : mint(0);
    }

    // Beware!! p[i] = k won't change the value of p.a[i]
    mint operator[](const int i) const {
        return (i < a.size() && i >= 0) ? a[i] : mint(0);
    }

    bool is_zero() const {
        for (int i = 0; i < size(); i++)
            if (a[i] != 0) return 0;
        return 1;
    }
    poly operator+(const poly &x) const {
        int n = max(size(), x.size());
        vector<mint> ans(n);
        for (int i = 0; i < n; i++) ans[i] = coef(i) + x.coef(i
            );
        while ((int)ans.size() && ans.back() == 0) ans.pop_back
            ();
        return ans;
    }
}
```

```
poly operator-(const poly &x) const {
    int n = max(size(), x.size());
    vector<mint> ans(n);
    for (int i = 0; i < n; i++) ans[i] = coef(i) - x.coef(i
        );
    while ((int)ans.size() && ans.back() == 0) ans.pop_back
        ();
    return ans;
}
poly operator*(const poly &b) const {
    if (is_zero() || b.is_zero()) return {};
    vector<mint> ans = multiply(a, b.a);
    while ((int)ans.size() && ans.back() == 0) ans.pop_back
        ();
    return ans;
}
poly operator*(const mint &x) const {
    int n = size();
    vector<mint> ans(n);
    for (int i = 0; i < n; i++) ans[i] = a[i] * x;
    return ans;
}
poly operator/(const mint &x) const { return (*this) * x.
    inv(); }
poly &operator+=(const poly &x) { return *this = (*this) +
    x; }
poly &operator-=(const poly &x) { return *this = (*this) -
    x; }
poly &operator*=(const poly &x) { return *this = (*this) *
    x; }
poly &operator*=(const mint &x) { return *this = (*this) *
    x; }
poly &operator/=(const mint &x) { return *this = (*this) /
    x; }
poly mod_xk(int k) const {
    // modulo by x^k
    return {a.begin(), a.begin() + min(k, size())};
}
poly mul_xk(int k) const {
    // multiply by x^k
    poly ans(*this);
    ans.a.insert(ans.a.begin(), k, 0);
    return ans;
}
poly div_xk(int k) const {
    // divide by x^k
    return vector<mint>(a.begin() + min(k, (int)a.size()),
        a.end());
}
poly substr(int l, int r) const {
    // return mod_xk(r).div_xk(l)
    l = min(l, size());
    r = min(r, size());
    return vector<mint>(a.begin() + l, a.begin() + r);
}
poly reverse_it(int n, bool rev = 0) const {
    // reverses and leaves only n terms
    poly ans(*this);
    if (rev) // if rev = 1 then tail goes to head
        ans.a.resize(max(n, (int)ans.a.size()));
    reverse(ans.a.begin(), ans.a.end());
    return ans.mod_xk(n);
}
poly differentiate() const {
    int n = size();
    vector<mint> ans(n);
    for (int i = 1; i < size(); i++) ans[i - 1] = coef(i) *
        i;
    return ans;
}
```

```

}
poly integrate() const {
    int n = size();
    vector<mint> ans(n + 1);
    for (int i = 0; i < size(); i++) ans[i + 1] = coef(i) /
        (i + 1);
    return ans;
}
poly inverse(int n) const {
    // 1 / p(x) % x^n, O(nlogn)
    assert(!is_zero());
    assert(a[0] != 0);
    poly ans(mint(1) / a[0]);
    for (int i = 1; i < n; i *= 2)
        ans = (ans * mint(2) - ans * ans * mod_xk(2 * i)).
            mod_xk(2 * i);
    return ans.mod_xk(n);
}
pair<poly, poly> divmod_slow(const poly &b) const {
    // when divisor or quotient is small
    vector<mint> A(a);
    vector<mint> ans;
    while (A.size() >= b.a.size()) {
        ans.push_back(A.back() / b.a.back());
        if (ans.back() != mint(0))
            for (size_t i = 0; i < b.a.size(); i++)
                A[A.size() - i - 1] -= ans.back() * b.a[b.a
                    .size() - i - 1];
        A.pop_back();
    }
    reverse(ans.begin(), ans.end());
    return {ans, A};
}
pair<poly, poly> divmod(const poly &b) const {
    // returns quotient and remainder of a mod b
    if (size() < b.size()) return {poly{0}, *this};
    int d = size() - b.size();
    if (min(d, b.size()) < 250) return divmod_slow(b);
    poly D = (reverse_it(d + 1) * b.reverse_it(d + 1).
        inverse(d + 1))
        .mod_xk(d + 1)
        .reverse_it(d + 1, 1);
    return {D, *this - (D * b)};
}
poly operator/(const poly &t) const { return divmod(t).
    first; }
poly operator%(const poly &t) const { return divmod(t).
    second; }
poly &operator/=(const poly &t) { return *this = divmod(t).
    first; }
poly &operator%=(const poly &t) { return *this = divmod(t).
    second; }
poly log(int n) const {
    // ln p(x) mod x^n
    assert(a[0] == 1);
    return (differentiate().mod_xk(n) * inverse(n)).
        integrate().mod_xk(n);
}
poly exp(int n) const {
    // e ^ p(x) mod x^n
    if (is_zero()) return {1};
    assert(a[0] == 0);
    poly ans({1});
    int i = 1;
    while (i < n) {
        poly C = ans.log(2 * i).div_xk(i) - substr(i, 2 * i
            );
        ans -= (ans * C).mod_xk(i).mul_xk(i);
        i *= 2;
    }
}

```

```

}
return ans.mod_xk(n);
}
// better for small k, k < 100000
poly pow(int k, int n) const {
    // p(x)^k mod x^n
    if (is_zero()) return *this;
    poly ans({1}), b = mod_xk(n);
    while (k) {
        if (k & 1) ans = (ans * b).mod_xk(n);
        b = (b * b).mod_xk(n);
        k >>= 1;
    }
    return ans;
}
int leading_xk() const {
    // minimum i such that C[i] > 0
    if (is_zero()) return 1000000000;
    int res = 0;
    while (a[res] == 0) res++;
    return res;
}
// better for k > 100000
poly pow2(int k, int n) const {
    // p(x)^k mod x^n
    if (is_zero()) return *this;
    int i = leading_xk();
    mint j = a[i];
    poly t = div_xk(i) / j;
    poly ans = (t.log(n) * mint(k)).exp(n);
    if (1LL * i * k > n) ans = {0};
    else ans = ans.mul_xk(i * k).mod_xk(n);
    ans *= (j.pwr(j, k));
    return ans;
}
// if the poly is not zero but the result is zero, then no
// solution
// poly sqrt(int n) const {
//     if ((*this)[0] == mint(0)) {
//         for (int i = 1; i < size(); i++) {
//             if ((*this)[i] != mint(0)) {
//                 if (i & 1) return {};
//                 if (n - i / 2 <= 0) break;
//                 return div_xk(i).sqrt(n - i / 2).mul_xk(i / 2);
//             }
//         }
//         return {};
//     }
//     mint s = (*this)[0].sqrt();
//     if (s == 0) return {};
//     poly y = *this / (*this)[0];
//     poly ret({1});
//     mint inv2 = mint(1) / 2;
//     for (int i = 1; i < n; i <= 1) {
//         ret = (ret + y.mod_xk(i <= 1) * ret.inverse(i <= 1))
//             * inv2;
//     }
//     return ret.mod_xk(n) * s;
// }
poly root(int n, int k = 2) const {
    // kth root of p(x) mod x^n
    if (is_zero()) return *this;
    if (k == 1) return mod_xk(n);
    assert(a[0] == 1);
    poly q({1});
    for (int i = 1; i < n; i <= 1) {
        if (k == 2) q += mod_xk(2 * i) * q.inverse(2 * i);
        else q = q * mint(k - 1) + mod_xk(2 * i) * q.
            inverse(2 * i).pow(k - 1, 2 * i);
    }
}

```

```

    q = q.mod_xk(2 * i) / mint(k);
}
return q.mod_xk(n);
}
poly mulx(mint x) {
    // component-wise multiplication with x^k
    mint cur = 1;
    poly ans(*this);
    for (int i = 0; i < size(); i++) ans.a[i] *= cur, cur
        *= x;
    return ans;
}
poly mulx_sq(mint x) {
    // component-wise multiplication with x^{k^2}
    mint cur = x, total = 1, xx = x * x;
    poly ans(*this);
    for (int i = 0; i < size(); i++) ans.a[i] *= total,
        total *= cur, cur *= xx;
    return ans;
}
vector<mint> chirpz_even(mint z, int n) {
    // P(1), P(z^2), P(z^4), ..., P(z^{2(n-1)})
    int m = size() - 1;
    if (is_zero()) return vector<mint>(n, 0);
    vector<mint> vv(m + n);
    mint iz = z.inv(), zz = iz * iz, cur = iz, total = 1;
    for (int i = 0; i <= max(n - 1, m); i++) {
        if (i <= m) vv[m - i] = total;
        if (i < n) vv[m + i] = total;
        total *= cur;
        cur *= zz;
    }
    poly w = (mulx_sq(z) * vv).substr(m, m + n).mulx_sq(z);
    vector<mint> ans(n);
    for (int i = 0; i < n; i++) ans[i] = w[i];
    return ans;
}
// O(nlogn)
vector<mint> chirpz(mint z, int n) {
    // P(1), P(z), P(z^2), ..., P(z^{n-1})
    auto even = chirpz_even(z, (n + 1) / 2);
    auto odd = mulx(z).chirpz_even(z, n / 2);
    vector<mint> ans(n);
    for (int i = 0; i < n / 2; i++) {
        ans[2 * i] = even[i];
        ans[2 * i + 1] = odd[i];
    }
    if (n % 2 == 1) ans[n - 1] = even.back();
    return ans;
}
poly shift_it(int m, vector<poly> &pw) {
    if (size() <= 1) return *this;
    while (m >= size()) m /= 2;
    poly q(a.begin() + m, a.end());
    return q.shift_it(m, pw) * pw[m] + mod_xk(m).shift_it(m
        , pw);
}
// n log(n)
poly shift(mint a) {
    // p(x + a)
    int n = size();
    if (n == 1) return *this;
    vector<poly> pw(n);
    pw[0] = poly({1});
    pw[1] = poly({a, 1});
    int i = 2;
    for (; i < n; i *= 2) pw[i] = pw[i / 2] * pw[i / 2];
    return shift_it(i, pw);
}

```

```

mint eval(mint x) {
    // evaluates in single point x
    mint ans(0);
    for (int i = size() - 1; i >= 0; i--) {
        ans *= x;
        ans += a[i];
    }
    return ans;
}
// p(g(x))
// O(n^2 log n)
poly composition_brute(poly g, int deg) {
    int n = size();
    poly c(deg, 0), pw({1});
    for (int i = 0; i < min(deg, n); i++) {
        int d = min(deg, (int)pw.size());
        for (int j = 0; j < d; j++) c.a[j] += coef(i) * pw[j];
        pw *= g;
        if (pw.size() > deg) pw.a.resize(deg);
    }
    return c;
}
// p(g(x))
// O(n log n * sqrt(n log n))
poly composition(poly g, int deg) {
    int n = size();
    int k = 1;
    while (k * k <= n) k++;
    k--;
    int d = n / k;
    if (k * d < n) d++;
    vector<poly> pw(k + 3, poly({1}));
    for (int i = 1; i <= k + 2; i++) {
        pw[i] = pw[i - 1] * g;
        if (pw[i].size() > deg) pw[i].a.resize(deg);
    }
    vector<poly> fi(k, poly(deg, 0));
    for (int i = 0; i < k; i++) {
        for (int j = 0; j < d; j++) {
            int idx = i * d + j;
            if (idx >= n) break;
            int sz = min(fi[i].size(), pw[j].size());
            for (int t = 0; t < sz; t++) fi[i].a[t] += pw[j][t] * coef(idx);
        }
    }
    poly ret(deg, 0), gd({1});
    for (int i = 0; i < k; i++) {
        fi[i] = fi[i] * gd;
        int sz = min((int)ret.size(), (int)fi[i].size());
        for (int j = 0; j < sz; j++) ret.a[j] += fi[i][j];
        gd = gd * pw[d];
        if (gd.size() > deg) gd.a.resize(deg);
    }
    return ret;
}
poly build(vector<poly> &ans, int v, int l, int r, vector<
    mint> &vec) {
    // builds evaluation tree for (x-a1)(x-a2)...(x-an)
    if (l == r) return ans[v] = poly({-vec[l], 1});
    int mid = l + r >> 1;
    return ans[v] = build(ans, 2 * v, l, mid, vec) *
        build(ans, 2 * v + 1, mid + 1, r, vec);
}
vector<mint> eval(vector<poly> &tree, int v, int l, int r,
    vector<mint> &vec) {
    // auxiliary evaluation function
    if (l == r) return {eval(vec[l])};

```

```

    if (size() < 400) {
        vector<mint> ans(r - l + 1, 0);
        for (int i = l; i <= r; i++) ans[i - l] = eval(vec[
            i]);
        return ans;
    }
    int mid = l + r >> 1;
    auto A = (*this % tree[2 * v]).eval(tree, 2 * v, l, mid
        , vec);
    auto B = (*this % tree[2 * v + 1]).eval(tree, 2 * v +
        1, mid + 1, r, vec);
    A.insert(A.end(), B.begin(), B.end());
    return A;
}
// O(n log^2 n)
vector<mint> eval(vector<mint> x) {
    // evaluate polynomial in (x_0, ..., x_{n-1})
    int n = x.size();
    if (is_zero()) return vector<mint>(n, mint(0));
    vector<poly> tree(4 * n);
    build(tree, 1, 0, n - 1, x);
    return eval(tree, 1, 0, n - 1, x);
}
poly interpolate(
    vector<poly> &tree, int v, int l, int r, int ly, int ry
        , vector<mint> &y
) {
    // auxiliary interpolation function
    if (l == r) return {y[ly] / a[0]};
    int mid = l + r >> 1;
    int midy = ly + ry >> 1;
    auto A = (*this % tree[2 * v]).interpolate(tree, 2 * v,
        l, mid, ly, midy, y);
    auto B = (*this % tree[2 * v + 1])
        .interpolate(tree, 2 * v + 1, mid + 1, r,
            midy + 1, ry, y);
    return A * tree[2 * v + 1] + B * tree[2 * v];
}
}
};

```

4.3 Convolutions

FFT.cpp

Description: Computa convolução (multiplicação) de polinômios em $O(N \log N)$, sendo N a soma dos graus dos polinômios. Testado e sem erros de precisão com polinômios de grau até $3 \cdot 10^5$ e constantes até 10^6 .
Time: $O(N \log N)$.

d41d8c, 67 lines

```

struct base {
    double a, b;
    base(double _a = 0, double _b = 0) : a(_a), b(_b) {}
    const base operator+(const base &c) const { return base(a +
        c.a, b + c.b); }
    const base operator-(const base &c) const { return base(a -
        c.a, b - c.b); }
    const base operator*(const base &c) const {
        return base(a * c.a - b * c.b, a * c.b + b * c.a);
    }
}

using poly = vector<base>;
const double PI = acos(-1);

void fft(poly &a, bool inv = 0) {
    int n = (int)a.size();

    for (int i = 0; i < n; i++) {
        int bit = n >> 1, j = 0, k = i;
        while (bit > 0) {
            if (k & 1) j += bit;

```

```

            k >>= 1, bit >>= 1;
        }
        if (i < j) swap(a[i], a[j]);
    }

    double angle = 2 * PI / n * (inv ? -1 : 1);
    poly wn(n / 2);
    for (int i = 0; i < n / 2; i++) wn[i] = {cos(angle * i),
        sin(angle * i)};

    for (int len = 2; len <= n; len <= 1) {
        int aux = len / 2;
        int step = n / len;
        for (int i = 0; i < n; i += len) {
            for (int j = 0; j < aux; j++) {
                base v = a[i + j + aux] * wn[step * j];
                a[i + j + aux] = a[i + j] - v;
                a[i + j] = a[i + j] + v;
            }
        }
    }

    for (int i = 0; inv && i < n; i++) a[i].a /= n, a[i].b /= n;
}

vector<ll> multiply(vector<ll> &ta, vector<ll> &tb) {
    int n = (int)ta.size(), m = (int)tb.size();
    int t = n + m - 1, sz = 1;
    while (sz < t) sz <= 1;

    poly a(sz), b(sz), c(sz);

    for (int i = 0; i < sz; i++) {
        a[i] = i < n ? base((double)ta[i]) : base(0);
        b[i] = i < m ? base((double)tb[i]) : base(0);
    }

    fft(a, 0), fft(b, 0);
    for (int i = 0; i < sz; i++) c[i] = a[i] * b[i];
    fft(c, 1);

    vector<ll> res(sz);
    for (int i = 0; i < sz; i++) res[i] = ll(round(c[i].a));

    while ((int)res.size() > t && res.back() == 0) res.pop_back
        ();

    return res;
}

```

NTT.cpp

d41d8c, 37 lines

```

template <auto MOD, typename T = Mint<MOD>>
void ntt(vector<T> &a, bool inv = 0) {
    int n = (int)a.size();
    auto b = a;
    T g = 1;
    while ((g ^ (MOD / 2)) == 1) g += 1;
    if (inv) g = T(1) / g;
    for (int step = n / 2; step; step /= 2) {
        T w = g ^ (MOD / (n / step)), wn = 1;
        for (int i = 0; i < n / 2; i += step) {
            for (int j = 0; j < step; j++) {
                auto u = a[2 * i + j], v = wn * a[2 * i + j +
                    step];
                b[i + j] = u + v;
                b[i + n / 2 + j] = u - v;
            }

```

```

        wn = wn * w;
    }
    swap(a, b);
}
if (inv) {
    T invn = T(1) / n;
    for (int i = 0; i < n; i++) a[i] *= invn;
}
}
}

```

```

template <auto MOD, typename T = Mint<MOD>>
vector<T> multiply(vector<T> a, vector<T> b) {
    int n = (int)a.size(), m = (int)b.size();
    int t = n + m - 1, sz = 1;
    while (sz < t) sz <= 1;
    a.resize(sz), b.resize(sz);
    ntt<MOD>(a, 0), ntt<MOD>(b, 0);
    for (int i = 0; i < sz; i++) a[i] *= b[i];
    ntt<MOD>(a, 1);
    while ((int)a.size() > t) a.pop_back();
    return a;
}

```

NTTBigModulo.cpp

d41d8c, 2 lines

```

const int MOD1 = 1004535809;
const int MOD2 = 1092616193;

```

Convolutions.cpp

Description: Fast Walsh Hadamard Transform, FWHT<OP> gives the transformation for the operation OP in c++ like syntax

Time: $\mathcal{O}(n \log(n))$

d41d8c, 108 lines

```

template<char op, class T> vector<T> FWHT(vector<T> f, bool inv
= false) {
    int n = f.size();
    for (int k = 0; (n-1)>>k; k++) for (int i = 0; i < n; i++) if
        (i>>k&1) {
        int j = i^(1<<k);
        if (op == '^') f[j] += f[i], f[i] = f[j] - 2*f[i];
        if (op == '|') f[i] += (inv ? -1 : 1) * f[j];
        if (op == '&') f[j] += (inv ? -1 : 1) * f[i];
    }
    if (op == '^' and inv) for (auto& i : f) i /= n;
    return f;
}

```

```

// Generalizacao de FWHT de Xor
//
// Convolucao de soma mod B, usar tamanho potencia de B!!
// Precisa definir o tipo T e a raiz primitiva g
// satisfazendo g^b == g
//
// Se possivel, hardcodar a multiplicacao de matriz
// feita em cada iteracao faz ficar bem mais rapido
//
// O(n b log_b(n))
// Exemplos da FWHT Generalizada:
//
// mod 7, resposta mod 998244353:
// T = mint, g = 14553391
//
// mod 3, resposta cabe em um long long:
// T = array<ll, 2>, g = {0, 1};
//
// using T = array<ll, 2>;
// T operator +(const T& a, const T& b) {
//     return T{a[0] + b[0], a[1] + b[1]};
// }

```

```

// T operator *(const T& a, const T& b) {
//     return T{a[0] * b[0] - a[1] * b[1],
//             a[0] * b[1] + a[1] * b[0] - a[1] * b[1]};
// };
// T operator /(const T& a, const int& b) {
//     return T{a[0] / b, a[1] / b};
// }

```

```

template<class T>
vector<T> FWHT(vector<T> f, int b, T g, bool inv = false) {
    int n = f.size();

    vector<T> w(b);
    w[1] = g;
    for (int i = 2; i < b; i++) w[i] = w[i - 1] * g;
    w[0] = w[b - 1] * g;

    if (inv) reverse(w.begin() + 1, w.end());

    for (int pot = 1; pot < n; pot *= b) {
        for (int i = 0; i < n; i++) if (!(i / pot % b)) {
            vector<T> res(b);
            for (int j = 0; j < b; j++) {
                for (int k = 0; k < b; k++)
                    res[j] = res[j] + w[j * k % b] * f[i + k * pot];
                if (inv) res[j] = res[j] / b;
            }
            for (int j = 0; j < b; j++) f[i + j * pot] = res[j];
        }
    }
    return f;
}

```

```

vector<mint> subset_convolution(vector<mint> A, vector<mint> B)
{
    int n = int(max(A.size(), B.size()));
    int N = 0;
    while ((1 << N) < n) N++;
    A.resize(1 << N), B.resize(1 << N);
    vector a(1 << N, vector<mint>(N + 1)), b(1 << N, vector<
        mint>(N + 1));
    for (int i = 0; i < 1 << N; i++) {
        int popcnt = __builtin_popcount(i);
        a[i][popcnt] = A[i];
        b[i][popcnt] = B[i];
    }
    for (int j = 0; j < N; j++) {
        for (int i = 0; i < 1 << N; i++) {
            if (~i >> j & 1) continue;
            for (int popcnt = 0; popcnt <= N; popcnt++) {
                a[i][popcnt] += a[i ^ (1 << j)][popcnt];
                b[i][popcnt] += b[i ^ (1 << j)][popcnt];
            }
        }
    }
    vector c(1 << N, vector<mint>(N + 1));
    for (int i = 0; i < 1 << N; i++) {
        for (int j = 0; j <= N; j++)
            for (int k = 0; k + j <= N; k++) c[i][j + k] += a[i
                ][j] * b[i][k];
    }
    for (int j = N - 1; j >= 0; j--) {
        for (int i = (1 << N) - 1; i >= 0; i--) {
            if (~i >> j & 1) continue;
            for (int popcnt = 0; popcnt <= N; popcnt++)
                c[i][popcnt] -= c[i ^ (1 << j)][popcnt];
        }
    }
}

```

```

vector<mint> ans(1 << N);
for (int i = 0; i < 1 << N; i++) {
    int popcnt = __builtin_popcount(i);
    ans[i] = c[i][popcnt];
}
return ans;
}

```

4.4 MIT Numerical

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum. The maximum error in the result is eps . Works equally well for maximization with a small change in the code. See TernarySearch.h in the Various chapter for a discrete version.

Usage: double func(double x) { return 4x+.3*x*x; }

double xmin = gss(-1000,1000,func);

Time: $\mathcal{O}(\log((b - a)/\epsilon))$

d41d8c, 14 lines

```

double gss(double a, double b, double (*f)(double)) {
    double r = (sqrt(5)-1)/2, eps = 1e-7;
    double x1 = b - r*(b-a), x2 = a + r*(b-a);
    double f1 = f(x1), f2 = f(x2);
    while (b-a > eps)
        if (f1 < f2) { //change to > to find maximum
            b = x2; x2 = x1; f2 = f1;
            x1 = b - r*(b-a); f1 = f(x1);
        } else {
            a = x1; x1 = x2; f1 = f2;
            x2 = a + r*(b-a); f2 = f(x2);
        }
    return a;
}

```

BerlekampMassey.h

Description: Recovers any n -order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.

Usage: BerlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}

"../number-theory/ModPow.h"

d41d8c, 20 lines

```

vector<ll> BerlekampMassey(vector<ll> s) {
    int n = sz(s), L = 0, m = 0;
    vector<ll> C(n), B(n), T;
    C[0] = B[0] = 1;

    ll b = 1;
    rep(i, 0, n) { ++m;
        ll d = s[i] % mod;
        rep(j, 1, L+1) d = (d + C[j] * s[i - j]) % mod;
        if (!d) continue;
        T = C; ll coef = d * modpow(b, mod-2) % mod;
        rep(j, m, n) C[j] = (C[j] - coef * B[j - m]) % mod;
        if (2 * L > i) continue;
        L = i + 1 - L; B = T; b = d; m = 0;
    }

    C.resize(L + 1); C.erase(C.begin());
    trav(x, C) x = (mod - x) % mod;
    return C;
}

```

LinearRecurrence.h

Description: Generates the k 'th term of an n -order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \dots n-1]$ and $tr[0 \dots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp-Massey.

Usage: linearRec({0, 1}, {1, 1}, k) // k 'th Fibonacci number

Time: $\mathcal{O}(n^2 \log k)$

d41d8c, 26 lines

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
    int n = sz(S);

    auto combine = [&](Poly a, Poly b) {
        Poly res(n * 2 + 1);
        rep(i,0,n+1) rep(j,0,n+1)
            res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
        for (int i = 2 * n; i > n; --i) rep(j,0,n)
            res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
        res.resize(n + 1);
        return res;
    };

    Poly pol(n + 1), e(pol);
    pol[0] = e[1] = 1;

    for (++k; k; k /= 2) {
        if (k % 2) pol = combine(pol, e);
        e = combine(e, e);
    }

    ll res = 0;
    rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
    return res;
}
```

Integrate.h
Description: Simple integration of a function over an interval using Simpson's rule. The error should be proportional to h^4 , although in practice you will want to verify that the result is stable to desired precision when epsilon changes.

d41d8c, 8 lines

```
double quad(double (*f)(double), double a, double b) {
    const int n = 1000;
    double h = (b - a) / 2 / n;
    double v = f(a) + f(b);
    rep(i,1,n*2)
        v += f(a + i*h) * (i&1 ? 4 : 2);
    return v * h / 3;
}
```

IntegrateAdaptive.h
Description: Fast integration using an adaptive Simpson's rule.
Usage: double z, y; double h(double x) { return x*x + y*y + z*z <= 1; } double g(double y) { ::y = y; return quad(h, -1, 1); } double f(double z) { ::z = z; return quad(g, -1, 1); } double sphereVol = quad(f, -1, 1), pi = sphereVol*3/4;

d41d8c, 16 lines

```
typedef double d;
d simpson(d (*f)(d), d a, d b) {
    d c = (a+b) / 2;
    return (f(a) + 4*f(c) + f(b)) * (b-a) / 6;
}
d rec(d (*f)(d), d a, d b, d eps, d S) {
    d c = (a+b) / 2;
    d S1 = simpson(f, a, c);
    d S2 = simpson(f, c, b), T = S1 + S2;
    if (abs (T - S) <= 15*eps || b-a < 1e-10)
        return T + (T - S) / 15;
    return rec(f, a, c, eps/2, S1) + rec(f, c, b, eps/2, S2);
}
d quad(d (*f)(d), d a, d b, d eps = 1e-8) {
    return rec(f, a, b, eps, simpson(f, a, b));
}
```

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.

Time: $\mathcal{O}(N^3)$

d41d8c, 15 lines

```
double det(vector<vector<double>>& a) {
    int n = sz(a); double res = 1;
    rep(i,0,n) {
        int b = i;
        rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
        if (i != b) swap(a[i], a[b]), res *= -1;
        res *= a[i][i];
        if (res == 0) return 0;
        rep(j,i+1,n) {
            double v = a[j][i] / a[i][i];
            if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
        }
    }
    return res;
}
```

IntDeterminant.h
Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.
Time: $\mathcal{O}(N^3)$

d41d8c, 18 lines

```
const ll mod = 12345;
ll det(vector<vector<ll>>& a) {
    int n = sz(a); ll ans = 1;
    rep(i,0,n) {
        rep(j,i+1,n) {
            while (a[j][i] != 0) { // gcd step
                ll t = a[i][i] / a[j][i];
                if (t) rep(k,i,n)
                    a[i][k] = (a[i][k] - a[j][k] * t) % mod;
                swap(a[i], a[j]);
                ans *= -1;
            }
        }
        ans = ans * a[i][i] % mod;
        if (!ans) return 0;
    }
    return (ans + mod) % mod;
}
```

Simplex.h
Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \leq b, x \geq 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.
Usage: vvd A = {{1,-1}, {-1,1}, {-1,-2}}; vvd b = {1,1,-4}, c = {-1,-1}, x; T val = LPSolver(A, b, c).solve(x);
Time: $\mathcal{O}(NM * \#pivots)$, where a pivot may be e.g. an edge relaxation.
 $\mathcal{O}(2^n)$ in the general case.

d41d8c, 68 lines

```
typedef double T; // long double, Rational, double + modP>...
typedef vector<T> vd;
typedef vector<vd> vvd;

const T eps = 1e-8, inf = 1/.0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s])) s=j

struct LPSolver {
    int m, n;
    vi N, B;
    vvd D;
```

d41d8c, 15 lines

```
LPSolver(const vvd& A, const vd& b, const vd& c) :
    m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
    rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
    rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] = b[i]; }
    rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
    N[n] = -1; D[m+1][n] = 1;
}

void pivot(int r, int s) {
    T *a = D[r].data(), inv = 1 / a[s];
    rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
        T *b = D[i].data(), inv2 = b[s] * inv;
        rep(j,0,n+2) b[j] -= a[j] * inv2;
        b[s] = a[s] * inv2;
    }
    rep(j,0,n+2) if (j != s) D[r][j] *= inv;
    rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
}

bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
        int s = -1;
        rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
        if (D[x][s] >= -eps) return true;
        int r = -1;
        rep(i,0,m) {
            if (D[i][s] <= eps) continue;
            if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
                < MP(D[r][n+1] / D[r][s], B[r])) r = i;
        }
        if (r == -1) return false;
        pivot(r, s);
    }
}

T solve(vd &x) {
    int r = 0;
    rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
    if (D[r][n+1] < -eps) {
        pivot(r, n);
        if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
        rep(i,0,m) if (B[i] == -1) {
            int s = 0;
            rep(j,1,n+1) ltj(D[i]);
            pivot(i, s);
        }
    }
    bool ok = simplex(1); x = vd(n);
    rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
    return ok ? D[m][n+1] : inf;
}
```

math-simplex.cpp
Description: Simplex algorithm. WARNING- segfaults on empty (size 0) max cx st Ax<=b, x>=0 do 2 phases; 1st check feasibility; 2nd check boundedness and ans

d41d8c, 40 lines

```
vector<double> simplex(vector<vector<double> > A, vector<double>
    > b, vector<double> c) {
    int n = (int) A.size(), m = (int) A[0].size()+1, r = n, s = m
        -1;
    vector<vector<double> > D = vector<vector<double> > (n+2,
        vector<double>(m+1));
    vector<int> ix = vector<int> (n+m);
```

```
for (int i=0; i<n+m; i++) ix[i] = i;
for (int i=0; i<n; i++) {
    for (int j=0; j<m-1; j++)D[i][j]=-A[i][j];
    D[i][m-1] = 1;
    D[i][m] = b[i];
    if (D[r][m] > D[i][m]) r = i;
}
for (int j=0; j<m-1; j++) D[n][j]=c[j];
D[n+1][m-1] = -1; int z = 0;
for (double d;;) {
    if (r < n) {
        swap(ix[s], ix[r+m]);
        D[r][s] = 1.0/D[r][s];
        for (int j=0; j<=m; j++) if (j!=s) D[r][j] *= -D[r][s];
        for(int i=0; i<=n+1; i++)if(i!=r) {
            for (int j=0; j<=m; j++) if(j!=s) D[i][j] += D[r][j] *
                D[i][s];
            D[i][s] *= D[r][s];
        }
        r = -1; s = -1;
        for (int j=0; j < m; j++) if (s<0 || ix[s]>ix[j]) {
            if (D[n+1][j]>eps || D[n+1][j]>-eps && D[n][j]>eps) s = j
                ;
        }
        if (s < 0) break;
        for (int i=0; i<n; i++) if(D[i][s]<-eps) {
            if (r < 0 || (d = D[r][m]/D[r][s]-D[i][m]/D[i][s]) < -eps
                || d < eps && ix[r+m] > ix[i+m]) r=i;
        }
        if (r < 0) return vector<double>(); // unbounded
    }
    if (D[n+1][m] < -eps) return vector<double>(); // infeasible
    vector<double> x(m-1);
    for (int i = m; i < n+m; i ++) if (ix[i] < m-1) x[ix[i]] = D[
        i-m][m];
    printf("%.21f\n", D[n][m]);
    return x; // ans: D[n][m]
}
```

SolveLinear.h
Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}(n^2m)$

```
typedef vector<double> vd;
const double eps = 1e-12;
```

```
int solveLinear(vector<vd>& A, vd& b, vd& x) {
    int n = sz(A), m = sz(x), rank = 0, br, bc;
    if (n) assert(sz(A[0]) == m);
    vi col(m); iota(all(col), 0);

    rep(i,0,n) {
        double v, bv = 0;
        rep(r,i,n) rep(c,i,m)
            if ((v = fabs(A[r][c])) > bv)
                br = r, bc = c, bv = v;
        if (bv <= eps) {
            rep(j,i,n) if (fabs(b[j]) > eps) return -1;
            break;
        }
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        rep(j,0,n) swap(A[j][i], A[j][bc]);
        bv = 1/A[i][i];
        rep(j,i+1,n) {
            double fac = A[j][i] * bv;
```

```
            b[j] -= fac * b[i];
            rep(k,i+1,m) A[j][k] -= fac*A[i][k];
        }
        rank++;
    }

    x.assign(m, 0);
    for (int i = rank; i--;) {
        b[i] /= A[i][i];
        x[col[i]] = b[i];
        rep(j,0,i) b[j] -= A[j][i] * b[i];
    }
    return rank; // (multiple solutions if rank < m)
}
```

SolveLinear2.h
Description: To get all uniquely determined values of x back from SolveLinear, make the following changes:

```
"SolveLinear.h"
d41d8c, 7 lines

rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
// ... then at the end:
x.assign(m, undefined);
rep(i,0,rank) {
    rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
    x[col[i]] = b[i] / A[i][i];
fail:; }
```

SolveLinearBinary.h
Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b .
Time: $\mathcal{O}(n^2m)$

```
typedef bitset<1000> bs;
```

```
int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
    int n = sz(A), rank = 0, br;
    assert(m <= sz(x));
    vi col(m); iota(all(col), 0);
    rep(i,0,n) {
        for (br=i; br<n; ++br) if (A[br].any()) break;
        if (br == n) {
            rep(j,i,n) if(b[j]) return -1;
            break;
        }
        int bc = (int)A[br]._Find_next(i-1);
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        rep(j,0,n) if (A[j][i] != A[j][bc]) {
            A[j].flip(i); A[j].flip(bc);
        }
        rep(j,i+1,n) if (A[j][i]) {
            b[j] ^= b[i];
            A[j] ^= A[i];
        }
        rank++;
    }

    x = bs();
    for (int i = rank; i--;) {
        if (!b[i]) continue;
        x[col[i]] = 1;
        rep(j,0,i) b[j] ^= A[j][i];
    }
    return rank; // (multiple solutions if rank < m)
}
```

MatrixInverse.h
Description: Invert matrix A . Returns rank; result is stored in A unless singular ($\text{rank} < n$). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of $A \pmod p$, and k is doubled in each step.
Time: $\mathcal{O}(n^3)$

```
int matInv(vector<vector<double>>& A) {
    int n = sz(A); vi col(n);
    vector<vector<double>> tmp(n, vector<double>(n));
    rep(i,0,n) tmp[i][i] = 1, col[i] = i;
```

```
    rep(i,0,n) {
        int r = i, c = i;
        rep(j,i,n) rep(k,i,n)
            if (fabs(A[j][k]) > fabs(A[r][c]))
                r = j, c = k;
        if (fabs(A[r][c]) < 1e-12) return i;
        A[i].swap(A[r]); tmp[i].swap(tmp[r]);
        rep(j,0,n)
            swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
        swap(col[i], col[c]);
        double v = A[i][i];
        rep(j,i+1,n) {
            double f = A[j][i] / v;
            A[j][i] = 0;
            rep(k,i+1,n) A[j][k] -= f*A[i][k];
            rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
        }
        rep(j,i+1,n) A[i][j] /= v;
        rep(j,0,n) tmp[i][j] /= v;
        A[i][i] = 1;
    }
}
```

```
for (int i = n-1; i > 0; --i) rep(j,0,i) {
    double v = A[j][i];
    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
}
```

```
    rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
    return n;
}
```

Tridiagonal.h
Description: $x = \text{tridiagonal}(d,p,q,b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i, 1 \leq i \leq n,$$
where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, \dots, -1, 1\}, \{0, c_1, c_2, \dots, c_n\}, \{b_1, b_2, \dots, b_n, 0\}, \{a_0, d_1, d_2, \dots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.
If $|d_i| > |p_i| + |q_{i-1}|$ for all i , or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.
Time: $\mathcal{O}(N)$

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>& super,
    const vector<T>& sub, vector<T> b) {
```

```
int n = sz(b); vi tr(n);
rep(i,0,n-1) {
    if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
        b[i+1] -= b[i] * diag[i+1] / super[i];
        if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
        diag[i+1] = sub[i]; tr[+i] = 1;
    } else {
        diag[i+1] -= super[i]*sub[i]/diag[i];
        b[i+1] -= b[i]*sub[i]/diag[i];
    }
}
for (int i = n; i--; ) {
    if (tr[i]) {
        swap(b[i], b[i-1]);
        diag[i-1] = diag[i];
        b[i] /= super[i-1];
    } else {
        b[i] /= diag[i];
        if (i) b[i-1] -= b[i]*super[i-1];
    }
}
return b;
}
```

Number theory (5)

5.1 Modular arithmetic

ModLog.h
Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a .
Time: $\mathcal{O}(\sqrt{m})$

```
ll modLog(ll a, ll b, ll m) {
    ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<ll, ll> A;
    while (j <= n && (e = f * e % m) != b % m)
        A[e * b % m] = j++;
    if (e == b % m) return j;
    if (__gcd(m, e) == __gcd(m, b))
        rep(i,2,n+2) if (A.count(e = e * f % m))
            return n * i - A[e];
    return -1;
}
```

ModSum.h
Description: Sums of mod'ed arithmetic progressions.
 $\text{modsum}(\text{to}, c, k, m) = \sum_{i=0}^{\text{to}-1} (ki + c) \% m$. divsum is similar but for floored division.
Time: $\log(m)$, with a large constant.

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }

ull divsum(ull to, ull c, ull k, ull m) {
    ull res = k / m * sumsq(to) + c / m * to;
    k %= m; c %= m;
    if (!k) return res;
    ull to2 = (to * k + c) / m;
    return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}

ll modsum(ull to, ll c, ll k, ll m) {
    c = ((c % m) + m) % m;
    k = ((k % m) + m) % m;
    return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h
Description: Calculate $a \cdot b \pmod c$ (or $a^b \pmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.
Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
    ll ret = a * b - M * ull(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {
    ull ans = 1;
    for (; e; b = modmul(b, b, mod), e /= 2)
        if (e & 1) ans = modmul(ans, b, mod);
    return ans;
}
```

ModSqrt.h
Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).
Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

```
"ModPow.h"
ll sqrt(ll a, ll p) {
    a %= p; if (a < 0) a += p;
    if (a == 0) return 0;
    assert(modpow(a, (p-1)/2, p) == 1); // else no solution
    if (p % 4 == 3) return modpow(a, (p+1)/4, p);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
    ll s = p - 1, n = 2;
    int r = 0, m;
    while (s % 2 == 0)
        ++r, s /= 2;
    while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
    ll x = modpow(a, (s + 1) / 2, p);
    ll b = modpow(a, s, p), g = modpow(n, s, p);
    for (;;) r = m) {
        ll t = b;
        for (m = 0; m < r && t != 1; ++m)
            t = t * t % p;
        if (m == 0) return x;
        ll gs = modpow(g, 1LL << (r - m - 1), p);
        g = gs * gs % p;
        x = x * gs % p;
        b = b * g % p;
    }
}
```

5.2 Primality

FastEratosthenes.h
Description: Prime sieve for generating all primes smaller than LIM.
Time: LIM=1e9 $\approx$ 1.5s

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
    const int S = (int)round(sqrt(LIM)), R = LIM / 2;
    vi pr = {2}, sieve(S+1); pr.reserve((int)(LIM/log(LIM)*1.1));
    vector<pii> cp;
    for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
        cp.push_back({i, i * i / 2});
        for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
    }
    for (int L = 1; L <= R; L += S) {
        array<bool, S> block{};
        for (auto &[p, idx] : cp)
            for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
        rep(i,0,min(S, R - L))
            if (!block[i]) pr.push_back((L + i) * 2 + 1);
    }
    for (int i : pr) isPrime[i] = 1;
}
```

```
return pr;
}
```

MillerRabin.h
Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \pmod c$.

```
"ModMulLL.h"
bool isPrime(ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
        s = __builtin_ctzll(n-1), d = n >> s;
    for (ull a : A) { // ^ count trailing zeroes
        ull p = modpow(a%n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
            p = modmul(p, p, n);
        if (p != n-1 && i != s) return 0;
    }
    return 1;
}
```

Factor.h
Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}(n^{1/4})$, less for numbers with small factors.

```
"ModMulLL.h", "MillerRabin.h"
ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return modmul(x, x, n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}
vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}
```

5.3 Divisibility

euclid.h
Description: Finds two integers x and y , such that $ax + by = \text{gcd}(a, b)$. If you just need gcd, use the built in `__gcd` instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

```
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a/b * x, d;
}
```

CRT.h
Description: Chinese Remainder Theorem.
`crt(a, m, b, n)` computes x such that $x \equiv a \pmod m, x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

```
"euclid.h"
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
```

```
assert((a - b) % g == 0); // else no solution
x = (b - a) % n * x % n / g * m + a;
return x < 0 ? x + m*n/g : x;
}
```

5.3.1 Bézout’s identity

For $a \neq 0, b \neq 0$, then $d = gcd(a,b)$ is the smallest positive integer for which there are integer solutions to

$ax + by = d$

If (x,y) is one solution, then all solutions are given by

$\left(x + \frac{kb}{gcd(a,b)}, y - \frac{ka}{gcd(a,b)}\right), \quad k \in \mathbb{Z}$

phiFunction.h
Description: Euler’s ϕ function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1, p$ prime $\Rightarrow \phi(p^k) = (p - 1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1}p_2^{k_2}...p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1}...(p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$. $\sum_{d|n} \phi(d) = n, \sum_{1 \leq k \leq n, gcd(k,n)=1} k = n\phi(n)/2, n > 1$
Euler’s thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Fermat’s little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p \forall a$.

```
const int LIM = 5000000;
int phi[LIM];
```

```
void calculatePhi() {
    rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
    for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
        for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

5.4 Fractions

ContinuedFractions.h
Description: Given N and a real number $x \geq 0$, finds the closest rational approximation p/q with $p, q \leq N$. It will obey $|p/q - x| \leq 1/qN$.
For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. (p_k/q_k alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞ ; if x is the root of a degree 2 polynomial the a ’s eventually become cyclic.
Time: $\mathcal{O}(\log N)$

```
typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximate(d x, ll N) {
    ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
    for (;;) {
        ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf),
            a = (ll)floor(y), b = min(a, lim),
            NP = b*P + LP, NQ = b*Q + LQ;
        if (a > b) {
            // If b > a/2, we have a semi-convergent that gives us a
            // better approximation; if b = a/2, we *may* have one.
            // Return {P, Q} here for a more canonical approximation.
            return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
                make_pair(NP, NQ) : make_pair(P, Q);
        }
        if (abs(y = 1/(y - (d)a)) > 3*N) {
            return {NP, NQ};
        }
        LP = P; P = NP;
        LQ = Q; Q = NQ;
    }
}
```

```
FracBinarySearch.h
Description: Given f and N, finds the smallest fraction p/q ∈ [0, 1] such
that f(p/q) is true, and p, q ≤ N. You may want to throw an exception from
f if it finds an exact solution, in which case N can be removed.
Usage: fracBS([](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}
Time: O(log(N))
d41d8c, 25 lines

struct Frac { ll p, q; };

template<class F>
Frac fracBS(F f, ll N) {
    bool dir = 1, A = 1, B = 1;
    Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
    if (f(lo)) return lo;
    assert(f(hi));
    while (A || B) {
        ll adv = 0, step = 1; // move hi if dir, else lo
        for (int si = 0; step; (step *= 2) >= si) {
            adv += step;
            Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
            if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
                adv -= step; si = 2;
            }
        }
        hi.p += lo.p * adv;
        hi.q += lo.q * adv;
        dir = !dir;
        swap(lo, hi);
        A = B; B = !!adv;
    }
    return dir ? hi : lo;
}
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$

with $m > n > 0, k > 0, m \perp n$, and either m or n even.

5.6 Primes

5.6.1 Left-truncatable primes

Prime numbers such that any suffix of them is also a prime:

33,333,31
357,686,312,646,216,567,629,137

5.6.2 MIT Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a , except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 Estimates

$\sum_{d|n} d = O(n \log \log n)$.

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n} \mu(d) = [n = 1]$ (very useful)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n)g(d)$

$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m)g(\lfloor \frac{n}{m} \rfloor)$

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h
Description: Permutation -> integer conversion. (Not order preserving.) Integer -> permutation can use a lookup table.
Time: $\mathcal{O}(n)$

```
int permToInt(vi& v) {
    int use = 0, i = 0, r = 0;
    for(int x:v) r = r * ++i + __builtin_popcount(use & ~(1<<x)),
        use |= 1 << x; // (note: minus, not ~!)
    return r;
}
```

6.1.2 Cycles

Let $g_S(n)$ be the number of n -permutations whose cycle lengths all belong to the set S . Then

$$\sum_{n=0}^\infty g_S(n) \frac{x^n}{n!} = \exp \left(\sum_{n \in S} \frac{x^n}{n} \right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n - 1)(D(n - 1) + D(n - 2)) = nD(n - 1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

6.1.4 Burnside’s lemma

Given a group G of symmetries and a set X , the number of elements of X up to symmetry equals

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts “configurations” (of some sort) of length n , we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n, k)) = \frac{1}{n} \sum_{k|n} f(k) \phi(n/k).$$

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \quad p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k - 1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	$\sim 2\text{e}5$	$\sim 2\text{e}8$

6.2.2 Lucas’ Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \dots + n_1 p + n_0$ and $m = m_k p^k + \dots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod p$.

6.2.3 Binomials

multinomial.h

Description: Computes $\binom{k_1 + \dots + k_n}{k_1, k_2, \dots, k_n} = \frac{(\sum k_i)!}{k_1! k_2! \dots k_n!}$. d41d8c, 6 lines

```
ll multinomial(vi& v) {
    ll c = 1, m = v.empty() ? 1 : v[0];
    rep(i, 1, sz(v)) rep(j, 0, v[i])
        c = c * ++m / (j+1);
    return c;
}
```

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able).

$$B[0, \dots] = [1, -\tfrac{1}{2}, \tfrac{1}{6}, 0, -\tfrac{1}{30}, 0, \tfrac{1}{42}, \dots]$$

Sums of powers:

$$\sum_{i=1}^n n^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x) dx - \sum_{k=1}^{\infty} \frac{B_k}{k!} f^{(k-1)}(m)$$

multinomial Dinitz

$$\approx \int_m^{\infty} f(x) dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n, k) = c(n - 1, k - 1) + (n - 1)c(n - 1, k), \quad c(0, 0) = 1$$

$$\sum_{k=0}^n c(n, k) x^k = x(x + 1) \dots (x + n - 1)$$

$$c(8, k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$$

$$c(n, 2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$$

6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j + 1)$, $k + 1$ j:s s.t. $\pi(j) \geq j$, k j:s s.t. $\pi(j) > j$.

$$E(n, k) = (n - k)E(n - 1, k - 1) + (k + 1)E(n - 1, k)$$

$$E(n, 0) = E(n, n - 1) = 1$$

$$E(n, k) = \sum_{j=0}^k (-1)^j \binom{n+1}{j} (k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n, k) = S(n - 1, k - 1) + kS(n - 1, k)$$

$$S(n, 1) = S(n, n) = 1$$

$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

6.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$ For p prime,

$$B(p^m + n) \equiv mB(n) + B(n + 1) \pmod p$$

6.3.6 Labeled unrooted trees

on n vertices: n^{n-2}

on k existing trees of size n_i : $n_1 n_2 \dots n_k n^{k-2}$

with degrees d_i : $(n - 2)! / ((d_1 - 1)! \dots (d_n - 1)!)$

6.3.7 Catalan numbers

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1, \quad C_{n+1} = \frac{2(2n+1)}{n+2} C_n, \quad C_{n+1} = \sum C_i C_{n-i}$$

$$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$$

- sub-diagonal monotone paths in an $n \times n$ grid.

- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n + 1$ leaves (0 or 2 children).
- ordered trees with $n + 1$ vertices.
- ways a convex polygon with $n + 2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Graph (7)

7.1 Network flow

Dinitz.cpp

Description: Flow algorithm with complexity $O(V^2 E)$. With $U = \max|\text{cap}|$: $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching.

```
d41d8c, 68 lines
struct dinitz {
    const bool scaling = false; // com scaling -> O(nm log(U)),
    int lim; // com constante alta
    struct edge {
        int to, cap, rev, flow;
        bool res;
        edge(int to_, int cap_, int rev_, bool res_)
            : to(to_), cap(cap_), rev(rev_), flow(0), res(res_) {}
    };

    vector<vector<edge>>> g;
    vector<int> lev, beg;
    ll F;
    dinitz(int n) : g(n), F(0) {}

    void add(int a, int b, int c) {
        g[a].emplace_back(b, c, g[b].size(), false);
        g[b].emplace_back(a, 0, g[a].size()-1, true);
    }
    bool bfs(int s, int t) {
        lev = vector<int>(g.size(), -1); lev[s] = 0;
        beg = vector<int>(g.size(), 0);
        queue<int> q; q.push(s);
        while (q.size()) {
            int u = q.front(); q.pop();
            for (auto& i : g[u]) {
                if (lev[i.to] != -1 or (i.flow == i.cap)) continue;
                if (scaling and i.cap - i.flow < lim) continue;
                lev[i.to] = lev[u] + 1;
                q.push(i.to);
            }
        }
        return lev[t] != -1;
    }
    int dfs(int v, int s, int f = INF) {
        if (!f or v == s) return f;
        for (int& i = beg[v]; i < g[v].size(); i++) {
            auto& e = g[v][i];
            if (lev[e.to] != lev[v] + 1) continue;
            int foi = dfs(e.to, s, min(f, e.cap - e.flow));
            if (!foi) continue;
            e.flow += foi, g[e.to][e.rev].flow -= foi;
            return foi;
        }
        return 0;
    }
    ll max_flow(int s, int t) {
        for (lim = scaling ? (1<<30) : 1; lim; lim /= 2)
            while (bfs(s, t)) while (int ff = dfs(s, t)) F += ff;
        return F;
    }
}
```

```
    }
};

// Recupera as arestas do corte s-t
vector<pair<int, int>> get_cut(dinitz& g, int s, int t) {
    g.max_flow(s, t);
    vector<pair<int, int>> cut;
    vector<int> vis(g.g.size(), 0), st = {s};
    vis[s] = 1;
    while (st.size()) {
        int u = st.back(); st.pop_back();
        for (auto e : g.g[u]) if (!vis[e.to] and e.flow < e.cap)
            vis[e.to] = 1, st.push_back(e.to);
    }
    for (int i = 0; i < g.g.size(); i++) for (auto e : g.g[i])
        if (vis[i] and !vis[e.to] and !e.res) cut.emplace_back(i, e.to);
    return cut;
}
```

MinCostMaxFlow.cpp

Description: $min_{cost} flow(s, t, f)$ computa o par $(fluxo, custo)$ com $max(fluxo) \leq f$ que tenha $min(custo) \leq min_{cost} flow(s, t)$: Fluxo máximo de custo mínimo de s pra t Se for um dag, da pra substituir o SPFA por uma DP pra naopagar $O(nm)$ no começo Se nao tiver aresta com custo negativo, nao precisa do SPFA
Time: $\mathcal{O}(nm + f * m \log n)$

d41d8c, 115 lines

```
template<typename T> struct mcmf {
    struct edge {
        int to, rev, flow, cap; // para, id da reversa, fluxo, capacidade
        bool res; // se eh reversa
        T cost; // custo da unidade de fluxo
        edge() : to(0), rev(0), flow(0), cap(0), cost(0), res(false) {}
        edge(int to_, int rev_, int flow_, int cap_, T cost_, bool res_) : to(to_), rev(rev_), flow(flow_), cap(cap_), res(res_), cost(cost_) {}
    };

    vector<vector<edge>> g;
    vector<int> par_idx, par;
    T inf;
    vector<T> dist;

    mcmf(int n) : g(n), par_idx(n), par(n), inf(numeric_limits<T>::max()/3) {}

    void add(int u, int v, int w, T cost) { // de u pra v com cap w e custo cost
        edge a = edge(v, g[v].size(), 0, w, cost, false);
        edge b = edge(u, g[u].size(), 0, 0, -cost, true);

        g[u].push_back(a);
        g[v].push_back(b);
    }

    vector<T> spfa(int s) { // nao precisa se nao tiver custo negativo
        deque<int> q;
        vector<bool> is_inside(g.size(), 0);
        dist = vector<T>(g.size(), inf);

        dist[s] = 0;
        q.push_back(s);
        is_inside[s] = true;
    }
};
```

```
while (!q.empty()) {
    int v = q.front();
    q.pop_front();
    is_inside[v] = false;

    for (int i = 0; i < g[v].size(); i++) {
        auto [to, rev, flow, cap, res, cost] = g[v][i];
        if (flow < cap and dist[v] + cost < dist[to]) {
            dist[to] = dist[v] + cost;

            if (is_inside[to]) continue;
            if (!q.empty() and dist[to] > dist[q.front()]) q.push_back(to);
            else q.push_front(to);
            is_inside[to] = true;
        }
    }
}
return dist;
}

bool dijkstra(int s, int t, vector<T>& pot) {
    priority_queue<pair<T, int>, vector<pair<T, int>>, greater<>> q;
    dist = vector<T>(g.size(), inf);
    dist[s] = 0;
    q.emplace(0, s);
    while (q.size()) {
        auto [d, v] = q.top();
        q.pop();
        if (dist[v] < d) continue;
        for (int i = 0; i < g[v].size(); i++) {
            auto [to, rev, flow, cap, res, cost] = g[v][i];
            cost += pot[v] - pot[to];
            if (flow < cap and dist[v] + cost < dist[to]) {
                dist[to] = dist[v] + cost;
                q.emplace(dist[to], to);
                par_idx[to] = i, par[to] = v;
            }
        }
    }
    return dist[t] < inf;
}

pair<int, T> min_cost_flow(int s, int t, int flow = INF) {
    vector<T> pot(g.size(), 0);
    pot = spfa(s); // mudar algoritmo de caminho minimo aqui

    int f = 0;
    T ret = 0;
    while (f < flow and dijkstra(s, t, pot)) {
        for (int i = 0; i < g.size(); i++)
            if (dist[i] < inf) pot[i] += dist[i];

        int mn_flow = flow - f, u = t;
        while (u != s) {
            mn_flow = min(mn_flow, g[par[u]][par_idx[u]].cap - g[par[u]][par_idx[u]].flow);
            u = par[u];
        }

        ret += pot[t] * mn_flow;

        u = t;
        while (u != s) {
            g[par[u]][par_idx[u]].flow += mn_flow;
            g[u][g[par[u]][par_idx[u]].rev].flow -= mn_flow;
            u = par[u];
        }
    }
}
```

```
        f += mn_flow;
    }

    return make_pair(f, ret);
}

// Opcional: retorna as arestas originais por onde passa flow = cap
vector<pair<int, int>> recover() {
    vector<pair<int, int>> used;
    for (int i = 0; i < g.size(); i++) for (edge e : g[i])
        if (e.flow == e.cap && !e.res) used.push_back({i, e.to});
    return used;
}
};
```

LowerBoundMaxFlow.cpp

Description: precisa do dinitz add(a, b, l, r) adiciona aresta de a pra b, onde precisa passar f de fluxo, $l \leq f \leq r$. add(a, b, c) adiciona aresta de a pra b com capacidade c.
Time: Mesma complexidade do dinitz

d41d8c, 36 lines

```
struct lb_max_flow : dinitz {
    vector<int> d;
    lb_max_flow(int n) : dinitz(n + 2), d(n, 0) {}
    void add(int a, int b, int l, int r) {
        d[a] -= l;
        d[b] += l;
        dinitz::add(a, b, r - l);
    }
    void add(int a, int b, int c) {
        dinitz::add(a, b, c);
    }
    bool has_circulation() {
        int n = d.size();

        ll cost = 0;
        for (int i = 0; i < n; i++) {
            if (d[i] > 0) {
                cost += d[i];
                dinitz::add(n, i, d[i]);
            } else if (d[i] < 0) {
                dinitz::add(i, n+1, -d[i]);
            }
        }

        return (dinitz::max_flow(n, n+1) == cost);
    }
    bool has_flow(int src, int snk) {
        dinitz::add(snk, src, INF);
        return has_circulation();
    }
    ll max_flow(int src, int snk) {
        if (!has_flow(src, snk)) return -1;
        dinitz::F = 0;
        return dinitz::max_flow(src, snk);
    }
};
```

PushRelabel.cpp

Description: Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.

Time: $\mathcal{O}(V^2 \sqrt{E})$

d41d8c, 51 lines

```
typedef ll Flow;
struct Edge {
    int dest, back;
```

```
Flow f, c;
};

struct PushRelabel {
    vector<vector<Edge>> g;
    vector<Flow> ec;
    vector<Edge*> cur;
    vector<vi> hs; vi H;
    PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}

    void add_edge(int s, int t, Flow cap, Flow rcap=0) {
        if (s == t) return;
        Edge a = {t, sz(g[t]), 0, cap};
        Edge b = {s, sz(g[s]), 0, rcap};
        g[s].push_back(a);
        g[t].push_back(b);
    }

    void add_flow(Edge& e, Flow f) {
        Edge &back = g[e.dest][e.back];
        if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
        e.f += f; e.c -= f; ec[e.dest] += f;
        back.f -= f; back.c += f; ec[back.dest] -= f;
    }

    Flow maxflow(int s, int t) {
        int v = sz(g); H[s] = v; ec[t] = 1;
        vi co(2*v); co[0] = v-1;
        rep(i,0,v) cur[i] = g[i].data();
        trav(e, g[s]) add_flow(e, e.c);

        for (int hi = 0;;) {
            while (hs[hi].empty()) if (!hi--) return -ec[s];
            int u = hs[hi].back(); hs[hi].pop_back();
            while (ec[u] > 0) // discharge u
                if (cur[u] == g[u].data() + sz(g[u])) {
                    H[u] = le9;
                    trav(e, g[u]) if (e.c && H[u] > H[e.dest]+1)
                        H[u] = H[e.dest]+1, cur[u] = &e;
                    if (++co[H[u]], !--co[hi] && hi < v)
                        rep(i,0,v) if (hi < H[i] && H[i] < v)
                            --co[H[i]], H[i] = v + 1;
                    hi = H[u];
                } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
                    add_flow(*cur[u], min(ec[u], cur[u]->c));
                else ++cur[u];
        }
    }
};
```

StoerWagner.cpp

Description: $O(n^3)$ but faster, 1 indexed

d41d8c, 56 lines

```
const int N = 155;

mt19937 rnd(chrono::steady_clock::now().time_since_epoch()).
count());
struct StoerWagner {
    int n;
    long long G[N][N], dis[N];
    int idx[N];
    bool vis[N];
    const long long inf = 1e18;
    StoerWagner() {}
    StoerWagner(int _n) {
        n = _n;
        memset(G, 0, sizeof G);
    }
    void add_edge(int u, int v, long long w) { //undirected edge,
        multiple edges are merged into one edge
```

```
if (u != v) {
    G[u][v] += w;
    G[v][u] += w;
}
}
long long solve() {
    long long ans = inf;
    for (int i = 0; i < n; ++ i) idx[i] = i + 1;
    shuffle(idx, idx + n, rnd);
    while (n > 1) {
        int t = 1, s = 0;
        for (int i = 1; i < n; ++ i) {
            dis[idx[i]] = G[idx[0]][idx[i]];
            if (dis[idx[i]] > dis[idx[t]]) t = i;
        }
        memset(vis, 0, sizeof vis);
        vis[idx[0]] = true;
        for (int i = 1; i < n; ++ i) {
            if (i == n - 1) {
                if (ans > dis[idx[t]]) ans = dis[idx[t]]; //idx[s] -
                    idx[t] is in two halves of the mincut
                if (ans == 0) return 0;
                for (int j = 0; j < n; ++ j) {
                    G[idx[s]][idx[j]] += G[idx[j]][idx[t]];
                    G[idx[j]][idx[s]] += G[idx[j]][idx[t]];
                }
                idx[t] = idx[-- n];
            }
            vis[idx[t]] = true;
            s = t;
            t = -1;
            for (int j = 1; j < n; ++ j) {
                if (!vis[idx[j]]) {
                    dis[idx[j]] += G[idx[s]][idx[j]];
                    if (t == -1 || dis[idx[t]] < dis[idx[j]]) t = j;
                }
            }
        }
        return ans;
    }
};
```

7.2 Matching

Hungarian.cpp

Description: Resolve o problema de assignment (matriz n por n). Colocar os valores da matriz em 'a' (pode < 0). assignment() retorna um par com o valor do assignment minimo, e a coluna escolhida por cada linha

Time: $O(n^3)$.

d41d8c, 42 lines

```
template<typename T> struct hungarian {
    int n;
    vector<vector<T>> a;
    vector<T> u, v;
    vector<int> p, way;
    T inf;

    hungarian(int n_) : n(n_), u(n+1), v(n+1), p(n+1), way(n+1) {
        a = vector<vector<T>>(n, vector<T>(n));
        inf = numeric_limits<T>::max();
    }
    pair<T, vector<int>> assignment() {
        for (int i = 1; i <= n; i++) {
            p[0] = i;
            int j0 = 0;
            vector<T> minv(n+1, inf);
            vector<int> used(n+1, 0);
            do {
                used[j0] = true;
```

```
int i0 = p[j0], j1 = -1;
T delta = inf;
for (int j = 1; j <= n; j++) if (!used[j]) {
    T cur = a[i0-1][j-1] - u[i0] - v[j];
    if (cur < minv[j]) minv[j] = cur, way[j] = j0;
    if (minv[j] < delta) delta = minv[j], j1 = j;
}
for (int j = 0; j <= n; j++)
    if (used[j]) u[p[j]] += delta, v[j] -= delta;
    else minv[j] -= delta;
j0 = j1;
} while (p[j0] != 0);
do {
    int j1 = way[j0];
    p[j0] = p[j1];
    j0 = j1;
} while (j0);
}
vector<int> ans(n);
for (int j = 1; j <= n; j++) ans[p[j]-1] = j-1;
return make_pair(-v[0], ans);
}
};
```

Blossom.cpp

Description: Maximum matching in general graph. Time: $O(n^3)$. If bipartite, $O(nm)$ and doesn't need 'contract'

d41d8c, 77 lines

```
vector<int> g[MAX];
int match[MAX]; // match[i] = com quem i esta matchzado ou -1
int n, pai[MAX], base[MAX], vis[MAX];
queue<int> q;

void contract(int u, int v, bool first = 1) {
    static vector<bool> blossom;
    static int l;
    if (first) {
        blossom = vector<bool>(n, 0);
        vector<bool> teve(n, 0);
        int k = u; l = v;
        while (1) {
            teve[k = base[k]] = 1;
            if (match[k] == -1) break;
            k = pai[match[k]];
        }
        while (!teve[l = base[l]]) l = pai[match[l]];
    }
    while (base[u] != l) {
        blossom[base[u]] = blossom[base[match[u]]] = 1;
        pai[u] = v;
        v = match[u];
        u = pai[match[u]];
    }
    if (!first) return;
    contract(v, u, 0);
    for (int i = 0; i < n; i++) if (blossom[base[i]]) {
        base[i] = l;
        if (!vis[i]) q.push(i);
        vis[i] = 1;
    }
}

int getpath(int s) {
    for (int i = 0; i < n; i++) base[i] = i, pai[i] = -1, vis[i]
        = 0;
    vis[s] = 1; q = queue<int>(); q.push(s);
    while (q.size()) {
        int u = q.front(); q.pop();
        for (int i : g[u]) {
```

```

    if (base[i] == base[u] or match[u] == i) continue;
    if (i == s or (match[i] != -1 and pai[match[i]] != -1))
        contract(u, i);
    else if (pai[i] == -1) {
        pai[i] = u;
        if (match[i] == -1) return i;
        i = match[i];
        vis[i] = 1; q.push(i);
    }
}
}
return -1;
}

int blossom() {
    int ans = 0;
    memset(match, -1, sizeof(match));
    for (int i = 0; i < n; i++) if (match[i] == -1)
        for (int j : g[i]) if (match[j] == -1) {
            match[i] = j;
            match[j] = i;
            ans++;
            break;
        }
    for (int i = 0; i < n; i++) if (match[i] == -1) {
        int j = getpath(i);
        if (j == -1) continue;
        ans++;
        while (j != -1) {
            int p = pai[j], pp = match[p];
            match[p] = j;
            match[j] = p;
            j = pp;
        }
    }
    return ans;
}
}

```

Kuhn.cpp

Description: maximum matching in bipartite graph. 'n' and 'm' are the number of vertices in each partition. add(i, j) to add an edge between vertex i in partition A and vertex j in partition B (i < n, j < m).

Time: $\mathcal{O}(|V| * |E|)$, expected as fast as dinitz

d41d8c, 61 lines

```

// Kuhn
//
// Computa matching maximo em grafo bipartido
// 'n' e 'm' sao quantos vertices tem em cada particao
// chamar add(i, j) para add aresta entre o cara i
// da particao A, e o cara j da particao B
// (entao i < n, j < m)
// Para recuperar o matching, basta olhar 'ma' e 'mb'
// 'recover' recupera o min vertex cover como um par de
// {caras da particao A, caras da particao B}
//
// O(|V| * |E|)
// Na pratica, parece rodar tao rapido quanto o Dinitz

mt19937 rng((int) chrono::steady_clock::now().time_since_epoch()
().count());

```

```

struct kuhn {
    int n, m;
    vector<vector<int>> g;
    vector<int> vis, ma, mb;

    kuhn(int n_, int m_) : n(n_), m(m_), g(n),
        vis(n+m), ma(n, -1), mb(m, -1) {}
}

```

```

void add(int a, int b) { g[a].push_back(b); }

bool dfs(int i) {
    vis[i] = 1;
    for (int j : g[i]) if (!vis[n+j]) {
        vis[n+j] = 1;
        if (mb[j] == -1 or dfs(mb[j])) {
            ma[i] = j, mb[j] = i;
            return true;
        }
    }
    return false;
}

int matching() {
    int ret = 0, aum = 1;
    for (auto& i : g) shuffle(i.begin(), i.end(), rng);
    while (aum) {
        for (int j = 0; j < m; j++) vis[n+j] = 0;
        aum = 0;
        for (int i = 0; i < n; i++)
            if (ma[i] == -1 and dfs(i)) ret++, aum = 1;
        return ret;
    }
}

pair<vector<int>, vector<int>> recover(kuhn& K) {
    K.matching();
    int n = K.n, m = K.m;
    for (int i = 0; i < n+m; i++) K.vis[i] = 0;
    for (int i = 0; i < n; i++) if (K.ma[i] == -1) K.dfs(i);
    vector<int> ca, cb;
    for (int i = 0; i < n; i++) if (!K.vis[i]) ca.push_back(i);
    for (int i = 0; i < m; i++) if (K.vis[n+i]) cb.push_back(i);
    return {ca, cb};
}

```

Gomory-Hu.cpp

Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.

Time: $\mathcal{O}(V)$ Flow Computations

"PushRelabel.h" d41d8c, 13 lines

```

typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
    vector<Edge> tree;
    vi par(N);
    rep(i, 1, N) {
        PushRelabel D(N); // Dinic also works
        for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
        tree.push_back({i, par[i], D.calc(i, par[i])});
        rep(j, i+1, N)
            if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
    }
    return tree;
}
}

```

7.3 Trees

LinkCutTree.cpp

d41d8c, 208 lines

```

const int N = 1e5 + 9;

struct node {
    int p = 0, c[2] = {0, 0}, pp = 0;
    bool flip = 0;
    int sz = 0, ssz = 0, vsz = 0; // sz -> aux tree size, ssz =
        subtree size in rep tree, vsz = virtual tree size
    long long val = 0, sum = 0, lazy = 0, subsum = 0, vsum = 0;
}

```

```

node() {}
node(int x) {
    val = x; sum = x;
    sz = 1; lazy = 0;
    ssz = 1; vsz = 0;
    subsum = x; vsum = 0;
}
};

struct LCT {
    vector<node> t;
    LCT() {}
    LCT(int n) : t(n + 1) {}

    // <independant splay tree code>
    int dir(int x, int y) { return t[x].c[1] == y; }
    void set(int x, int d, int y) {
        if (x) t[x].c[d] = y, pull(x);
        if (y) t[y].p = x;
    }
    void pull(int x) {
        if (!x) return;
        int &l = t[x].c[0], &r = t[x].c[1];
        push(l); push(r);
        t[x].sum = t[l].sum + t[r].sum + t[x].val;
        t[x].sz = t[l].sz + t[r].sz + 1;
        t[x].ssz = t[l].ssz + t[r].ssz + t[x].vsz + 1;
        t[x].subsum = t[l].subsum + t[r].subsum + t[x].vsum + t[x].val;
    }
    void push(int x) {
        if (!x) return;
        int &l = t[x].c[0], &r = t[x].c[1];
        if (t[x].flip) {
            swap(l, r);
            if (l) t[l].flip ^= 1;
            if (r) t[r].flip ^= 1;
            t[x].flip = 0;
        }
        if (t[x].lazy) {
            t[x].val += t[x].lazy;
            t[x].sum += t[x].lazy * t[x].sz;
            t[x].subsum += t[x].lazy * t[x].ssz;
            t[x].vsum += t[x].lazy * t[x].vsz;
            if (l) t[l].lazy += t[x].lazy;
            if (r) t[r].lazy += t[x].lazy;
            t[x].lazy = 0;
        }
    }
    void rotate(int x, int d) {
        int y = t[x].p, z = t[y].p, w = t[x].c[d];
        swap(t[x].pp, t[y].pp);
        set(y, !d, w);
        set(x, d, y);
        set(z, dir(z, y), x);
    }
    void splay(int x) {
        for (push(x); t[x].p;) {
            int y = t[x].p, z = t[y].p;
            push(z); push(y); push(x);
            int dx = dir(y, x), dy = dir(z, y);
            if (!z) rotate(x, !dx);
            else if (dx == dy) rotate(y, !dx), rotate(x, !dx);
            else rotate(x, dy), rotate(x, dx);
        }
    }
    // </independant splay tree code>

    // making it a root in the rep. tree
    void make_root(int u) {

```

```

    access(u);
    int l = t[u].c[0];
    t[l].flip ^= 1;
    swap(t[l].p, t[l].pp);
    t[u].vsz += t[l].ssz;
    t[u].vsum += t[l].subsum;
    set(u, 0, 0);
}
// make the path from root to u a preferred path
// returns last path-parent of a node as it moves up the tree
int access(int _u) {
    int last = _u;
    for (int v = 0, u = _u; u; u = t[v = u].pp) {
        splay(u); splay(v);
        t[u].vsz -= t[v].ssz;
        t[u].vsum -= t[v].subsum;
        int r = t[u].c[1];
        t[u].vsz += t[r].ssz;
        t[u].vsum += t[r].subsum;
        t[v].pp = 0;
        swap(t[r].p, t[r].pp);
        set(u, 1, v);
        last = u;
    }
    splay(_u);
    return last;
}
void link(int u, int v) { // u → v
    // assert(!connected(u, v));
    make_root(v);
    access(u); splay(u);
    t[v].pp = u;
    t[u].vsz += t[v].ssz;
    t[u].vsum += t[v].subsum;
}
void cut(int u) { // cut par[u] → u, u is non root vertex
    access(u);
    assert(t[u].c[0] != 0);
    t[t[u].c[0]].p = 0;
    t[u].c[0] = 0;
    pull(u);
}
// parent of u in the rep. tree
int get_parent(int u) {
    access(u); splay(u); push(u);
    u = t[u].c[0]; push(u);
    while (t[u].c[1]) {
        u = t[u].c[1]; push(u);
    }
    splay(u);
    return u;
}
// root of the rep. tree containing this node
int find_root(int u) {
    access(u); splay(u); push(u);
    while (t[u].c[0]) {
        u = t[u].c[0]; push(u);
    }
    splay(u);
    return u;
}
bool connected(int u, int v) {
    return find_root(u) == find_root(v);
}
// depth in the rep. tree
int depth(int u) {
    access(u); splay(u);
    return t[u].sz;
}
}

```

```

int lca(int u, int v) {
    // assert(connected(u, v));
    if (u == v) return u;
    if (depth(u) > depth(v)) swap(u, v);
    access(v);
    return access(u);
}
int is_root(int u) {
    return get_parent(u) == 0;
}
int component_size(int u) {
    return t[find_root(u)].ssz;
}
int subtree_size(int u) {
    int p = get_parent(u);
    if (p == 0) {
        return component_size(u);
    }
    cut(u);
    int ans = component_size(u);
    link(p, u);
    return ans;
}
long long component_sum(int u) {
    return t[find_root(u)].subsum;
}
long long subtree_sum(int u) {
    int p = get_parent(u);
    if (p == 0) {
        return component_sum(u);
    }
    cut(u);
    long long ans = component_sum(u);
    link(p, u);
    return ans;
}
// sum of the subtree of u when root is specified
long long subtree_query(int u, int root) {
    int cur = find_root(u);
    make_root(root);
    long long ans = subtree_sum(u);
    make_root(cur);
    return ans;
}
// path sum
long long query(int u, int v) {
    int cur = find_root(u);
    make_root(u); access(v);
    long long ans = t[v].sum;
    make_root(cur);
    return ans;
}
void upd(int u, int x) {
    access(u); splay(u);
    t[u].val += x;
}
// add x to the nodes on the path from u to v
void upd(int u, int v, int x) {
    int cur = find_root(u);
    make_root(u); access(v);
    t[v].lazy += x;
    make_root(cur);
}
}t[2];

```

Block-Cut-Tree.cpp

Description: Block cut tree

Usage: art[i] responde o numero de novas componentes conexas criadas apos a remocao de i do grafo g
Se art[i] >= 1, i eh ponto de articulacao
Para todo i < blocks.size()
blocks[i] eh uma componente 2-vertice-conexa maximal
edgblocks[i] sao as arestas do bloco i
tree[i] eh um vertice da arvore que corresponde ao bloco i
Time: $O(n + m)$

d41d8c, 58 lines

```

struct block_cut_tree {
    vector<vector<int>> g, blocks, tree;
    vector<vector<pair<int, int>>> edgblocks;
    stack<int> s;
    stack<pair<int, int>> s2;
    vector<int> id, art, pos;

    block_cut_tree(vector<vector<int>> g_) : g(g_) {
        int n = g.size();
        id.resize(n, -1), art.resize(n), pos.resize(n);
        build();
    }

    int dfs(int i, int& t, int p = -1) {
        int lo = id[i] = t++;
        s.push(i);

        if (p != -1) s2.emplace(i, p);
        for (int j : g[i]) if (j != p and id[j] != -1) s2.emplace(i, j);

        for (int j : g[i]) if (j != p) {
            if (id[j] == -1) {
                int val = dfs(j, t, i);
                lo = min(lo, val);

                if (val >= id[i]) {
                    art[i]++;
                    blocks.emplace_back(1, i);
                    while (blocks.back().back() != j)
                        blocks.back().push_back(s.top()), s.pop();

                    edgblocks.emplace_back(1, s2.top()), s2.pop();
                    while (edgblocks.back().back() != pair(j, i))
                        edgblocks.back().push_back(s2.top()), s2.pop();
                }
                // if (val > id[i]) aresta i-j eh ponte
            }
            else lo = min(lo, id[j]);
        }

        if (p == -1 and art[i]) art[i]--;
        return lo;
    }

    void build() {
        int t = 0;
        for (int i = 0; i < g.size(); i++) if (id[i] == -1) dfs(i, t, -1);

        tree.resize(blocks.size());
        for (int i = 0; i < g.size(); i++) if (art[i])
            pos[i] = tree.size(), tree.emplace_back();

        for (int i = 0; i < blocks.size(); i++) for (int j : blocks[i]) {
            if (!art[j]) pos[j] = i;
            else tree[i].push_back(pos[j]), tree[pos[j]].push_back(i);
        }
    }
}

```

```
    }
};
```

DirectedMST.cpp

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.

Time: $\mathcal{O}(E \log V)$

```
"../data-structures/UnionFindRollback.h" d41d8c, 60 lines

struct Edge { int a, b; ll w; };
struct Node {
    Edge key;
    Node *l, *r;
    ll delta;
    void prop() {
        key.w += delta;
        if (l) l->delta += delta;
        if (r) r->delta += delta;
        delta = 0;
    }
    Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
    if (!a || !b) return a ? b;
    a->prop(), b->prop();
    if (a->key.w > b->key.w) swap(a, b);
    swap(a->l, (a->r = merge(b, a->r)));
    return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

pair<ll, vi> dmst(int n, int r, vector<Edge*& g) {
    RollbackUF uf(n);
    vector<Node*> heap(n);
    for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
    ll res = 0;
    vi seen(n, -1), path(n), par(n);
    seen[r] = r;
    vector<Edge> Q(n, in(n, {-1,-1})), comp;
    deque<tuple<int, int, vector<Edge>>> cys;
    rep(s,0,n) {
        int u = s, qi = 0, w;
        while (seen[u] < 0) {
            if (!heap[u]) return {-1,{};};
            Edge e = heap[u]->top();
            heap[u]->delta -= e.w, pop(heap[u]);
            Q[qi] = e, path[qi++] = u, seen[u] = s;
            res += e.w, u = uf.find(e.a);
            if (seen[u] == s) {
                Node* cyc = 0;
                int end = qi, time = uf.time();
                do cyc = merge(cyc, heap[w = path[--qi]]);
                while (uf.join(u, w));
                u = uf.find(u), heap[u] = cyc, seen[u] = -1;
                cys.push_front({u, time, {&Q[qi], &Q[end]}});
            }
        }
        rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
    }

    for (auto& [u,t,comp] : cys) { // restore sol (optional)
        uf.rollback(t);
        Edge inEdge = in[u];
        for (auto& e : comp) in[uf.find(e.b)] = e;
        in[uf.find(inEdge.b)] = inEdge;
    }
    rep(i,0,n) par[i] = in[i].a;
    return {res, par};
}
```

SteinerTree.cpp

Description: retorna o peso da menor arvore que cobre os vertices S. get steiner retorna o valor minimo e as arestas de uma solucao se nao tiver solucao retorna LINF otimizacao: joga um vertice x do S fora e pegue a resposta em dp(...)(x) e reconstrua a arvore a partir dele ta comentado no codigo as mudancas necessarias

Time: $\mathcal{O}\left(3^k * n + 2^k * m \log m\right)$

```
d41d8c, 68 lines

int n; // numero de vertices
vector<pair<int, int>> g[MAX]; // {vizinho, peso}
ll d[l << K][MAX]; // dp[mask][v] = arvore minima com o
                    subconjunto mask de S e o vertice v
ll vw[MAX]; // peso do vertice

ll steiner(const vector<int> &S) {
    int k = S.size(); // k—;
    for (int mask = 0; mask < (1 << k); mask++) for (int v = 0; v
        < n; v++) d[mask][v] = LINF;
    for (int v = 0; v < n; v++) d[0][v] = vw[v];
    for (int i = 0; i < k; ++i) d[l << i][S[i]] = vw[S[i]];
    for (int mask = 1; mask < (1 << k); mask++) {
        for (int a = (mask - 1) & mask; a; a = (a - 1) & mask) {
            int b = mask ^ a;
            if (b > a) break;
            for (int v = 0; v < n; v++)
                d[mask][v] = min(d[mask][v], d[a][v] + d[b][v] - vw[v])
                ;
        }
        priority_queue<pair<ll, int>> pq;
        for (int v = 0; v < n; v++) {
            if (d[mask][v] == LINF) continue;
            pq.emplace(-d[mask][v], v);
        }
        while (pq.size()) {
            auto [ndist, u] = pq.top(); pq.pop();
            if (-ndist > d[mask][u]) continue;
            for (auto [idx, w] : g[u]) if (d[mask][idx] > d[mask][u]
                + w + vw[idx]) {
                d[mask][idx] = d[mask][u] + w + vw[idx];
                pq.emplace(-d[mask][idx], idx);
            }
        }
    }
    return d[(1 << k) - 1][S[0]]; // S[k]
}

#warning se k=1 a solucao eh a folha isolada e a funcao retorna
    edg = {}
#warning se k=0 crasha
pair<ll,vector<pair<int,int>>> get_steiner(const vector<int> &S
    ) {
    int k = S.size(); // k—;
    ll ans = steiner(S);
    vector<pair<int,int>> edg;
    stack<pair<int,int>> stk;
    stk.emplace((1 << k) - 1, S[0]); // S[k]
    while (!stk.empty()) {
        bool cont = 0;
        auto [mask,u] = stk.top();stk.pop();
        if ((__builtin_popcount(mask) == 1 and u == S[__bit_width(
            mask) - 1])) continue;
        for (auto [idx, w] : g[u]){
            if (d[mask][u] == d[mask][idx] + w + vw[u]) {
                edg.emplace_back(u, idx);
                stk.emplace(mask, idx);
                cont = true;
            }
            break;
        }
    }
}
```

```
if (cont) continue;
for (int a = (mask - 1) & mask; a; a = (a - 1) & mask) {
    int b = mask ^ a;
    if (d[mask][u] == d[a][u] + d[b][u] - vw[u]) {
        stk.emplace(a, u);
        stk.emplace(b, u);
        cont = true;
        break;
    }
}
assert(!mask || cont);
}
return {ans, edg};
}
```

7.4 Math

7.4.1 Number of Spanning Trees

Create an $N \times N$ matrix mat , and for each edge $a \rightarrow b \in G$, do $mat[a][b]--$, $mat[b][b]++$ (and $mat[b][a]--$, $mat[a][a]++$ if G is undirected). Remove the i th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.4.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Geometry (8)

8.1 Geometric primitives

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

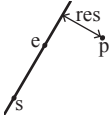
```
d41d8c, 28 lines

template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
    P operator+(P p) const { return P(x+p.x, y+p.y); }
    P operator-(P p) const { return P(x-p.x, y-p.y); }
    P operator*(T d) const { return P(x*d, y*d); }
    P operator/(T d) const { return P(x/d, y/d); }
    T dot(P p) const { return x*p.x + y*p.y; }
    T cross(P p) const { return x*p.y - y*p.x; }
    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x*x + y*y; }
    double dist() const { return sqrt((double)dist2()); }
    // angle to x-axis in interval [-pi, pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this/dist(); } // makes dist()==1
    P perp() const { return P(-y, x); } // rotates +90 degrees
    P normal() const { return perp().unit(); }
    // returns point rotated 'a' radians ccw around the origin
}
```

```
P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << ", " << p.y << ")"; }
};
```

lineDistance.h

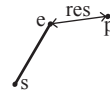
Description:
Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.



```
template<class P>
double lineDist(const P& a, const P& b, const P& p) {
    return (double) (b-a).cross(p-a)/(b-a).dist();
}
```

SegmentDistance.h

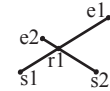
Description:
Returns the shortest distance between point p and the line segment from point s to e.
Usage: Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;



```
"Point.h"
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
    if (s==e) return (p-s).dist();
    auto d = (e-s).dist2(), t = min(d,max(.0, (p-s).dot(e-s)));
    return ((p-s)*d-(e-s)*t).dist()/d;
}
```

SegmentIntersection.h

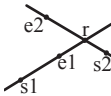
Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.
Usage: vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;



```
"Point.h", "OnSegment.h"
template<class P> vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b),
        oc = a.cross(b, c), od = a.cross(b, d);
    // Checks if intersection is single non-endpoint point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
    if (onSegment(a, b, d)) s.insert(d);
    return {all(s)};
}
```

lineIntersection.h

Description:
If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.
Usage: auto res = lineInter(s1,e1,s2,e2);
if (res.first == 1)
cout << "intersection point at " << res.second << endl;



```
"Point.h"
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return {-(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
```

sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 ⇔ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.
Usage: bool left = sideOf(p1,p2,q)==1;

```
"Point.h"
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }
```

```
template<class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
    auto a = (e-s).cross(p-s);
    double l = (e-s).dist()*eps;
    return (a > l) - (a < -l);
}
```

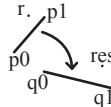
OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p)<=epsilon) instead when using Point<double>.

```
"Point.h"
template<class P> bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

linearTransformation.h

Description:
Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.



```
"Point.h"
typedef Point<double> P;
P linearTransformation(const P& p0, const P& p1,
    const P& q0, const P& q1, const P& r) {
    P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
    return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}
```

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage: vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }
// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i

```
d41d8c, 35 lines
struct Angle {
    int x, y;
    int t;
    Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
    Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
    int half() const {
        assert(x || y);
        return y < 0 || (y == 0 && x < 0);
    }
    Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
    Angle t180() const { return {-x, -y, t + half()}; }
    Angle t360() const { return {x, y, t + 1}; }
};
bool operator<(Angle a, Angle b) {
    // add a.dist2() and b.dist2() to also compare distances
    return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
        make_tuple(b.t, b.half(), a.x * (ll)b.y);
}
```

```
// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
    if (b < a) swap(a, b);
    return (b < a.t180() ?
        make_pair(a, b) : make_pair(b, a.t360()));
}
Angle operator+(Angle a, Angle b) { // point a + vector b
    Angle r(a.x + b.x, a.y + b.y, a.t);
    if (a.t180() < r) r.t--;
    return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
    int tu = b.t - a.t; a.t = b.t;
    return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
}
```

8.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

```
"Point.h"
typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) {
    if (a == b) { assert(r1 != r2); return false; }
    P vec = b - a;
    double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
        p = (d2 + r1*r1 - r2*r2)/(d2+2), h2 = r1*r1 - p*p*d2;
    if (sum*sum < d2 || dif*dif > d2) return false;
    P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
    *out = {mid + per, mid - per};
    return true;
}
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

```
"Point.h"
template<class P>
```

```
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
    P d = c2 - c1;
    double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0) return {};
    vector<pair<P, P>> out;
    for (double sign : {-1, 1}) {
        P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
        out.push_back({c1 + v * r1, c2 + v * r2});
    }
    if (h2 == 0) out.pop_back();
    return out;
}
```

CirclePolygonIntersection.h

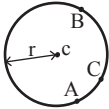
Description: Returns the area of the intersection of a circle with a ccw polygon.
Time: $\mathcal{O}(n)$

```
"../..content/geometry/Point.h"
d41d8c, 19 lines

typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
    auto tri = [&](P p, P q) {
        auto r2 = r * r / 2;
        P d = q - p;
        auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2();
        auto det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
        if (t < 0 || 1 <= s) return arg(p, q) * r2;
        P u = p + d * s, v = p + d * t;
        return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
    };
    auto sum = 0.0;
    rep(i,0,sz(ps))
        sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
    return sum;
}
```

circumcircle.h

Description:
The circumcircle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



```
"Point.h"
d41d8c, 9 lines

typedef Point<double> P;
double ccRadius(const P& A, const P& B, const P& C) {
    return (B-A).dist()* (C-B).dist()* (A-C).dist() /
        abs((B-A).cross(C-A))/2;
}

P ccCenter(const P& A, const P& B, const P& C) {
    P b = C-A, c = B-A;
    return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.
Time: expected $\mathcal{O}(n)$

```
"circumcircle.h"
d41d8c, 17 lines

pair<P, double> mec(vector<P> ps) {
    shuffle(all(ps), mt19937(time(0)));
    P o = ps[0];
    double r = 0, EPS = 1 + 1e-8;
    rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
        o = ps[i], r = 0;
        rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
            o = (ps[i] + ps[j]) / 2;

```

```
        r = (o - ps[i]).dist();
        rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
            o = ccCenter(ps[i], ps[j], ps[k]);
            r = (o - ps[i]).dist();
        }
    }
    return {o, r};
}
```

8.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.
Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}};
bool in = inPolygon(v, P{3, 3}, false);
Time: $\mathcal{O}(n)$

```
"Point.h", "OnSegment.h", "SegmentDistance.h"
d41d8c, 11 lines

template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
    int cnt = 0, n = sz(p);
    rep(i,0,n) {
        P q = p[(i + 1) % n];
        if (onSegment(p[i], q, a)) return !strict;
        //or: if (segDist(p[i], q, a) <= eps) return !strict;
        cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
    }
    return cnt;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

```
"Point.h"
d41d8c, 6 lines

template<class T>
T polygonArea2(vector<Point<T>>& v) {
    T a = v.back().cross(v[0]);
    rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
    return a;
}
```

PolygonCenter.h

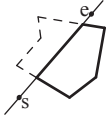
Description: Returns the center of mass for a polygon.
Time: $\mathcal{O}(n)$

```
"Point.h"
d41d8c, 9 lines

typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
    P res(0, 0); double A = 0;
    for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
        res = res + (v[i] + v[j]) * v[j].cross(v[i]);
        A += v[j].cross(v[i]);
    }
    return res / A / 3;
}
```

PolygonCut.h

Description:
Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.



Usage: vector<P> p = ...;
p = polygonCut(p, P(0,0), P(1,0));

```
"Point.h", "lineIntersection.h"
d41d8c, 13 lines

typedef Point<double> P;
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
    vector<P> res;

```

```
        rep(i,0,sz(poly)) {
            P cur = poly[i], prev = i ? poly[i-1] : poly.back();
            bool side = s.cross(e, cur) < 0;
            if (side != (s.cross(e, prev) < 0))
                res.push_back(lineInter(s, e, cur, prev).second);
            if (side)
                res.push_back(cur);
        }
        return res;
    }
}
```

ConvexHull.h

Description:
Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.
Time: $\mathcal{O}(n \log n)$



```
"Point.h"
d41d8c, 13 lines

typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
    if (sz(pts) <= 1) return pts;
    sort(all(pts));
    vector<P> h(sz(pts)+1);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t, reverse(all(pts)))
        for (P p : pts) {
            while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
            h[t++] = p;
        }
    return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).
Time: $\mathcal{O}(n)$

```
"Point.h"
d41d8c, 12 lines

typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
    int n = sz(S), j = n < 2 ? 0 : 1;
    pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
    rep(i,0,j)
        for (;;) j = (j + 1) % n {
            res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
            if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
                break;
        }
    return res.second;
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.
Time: $\mathcal{O}(\log N)$

```
"Point.h", "sideOf.h", "OnSegment.h"
d41d8c, 14 lines

typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true) {
    int a = 1, b = sz(l) - 1, r = !strict;
    if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
    if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
    if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p) <= -r)
        return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
    }
}
```

```
    return sgn(l[a].cross(l[b], p)) < r;
}
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: • (−1, −1) if no collision, • (i, −1) if touching the corner i, • (i, i) if along side (i, i + 1), • (i, j) if crossing sides (i, i + 1) and (j, j + 1). In the last case, if a corner i is crossed, this is treated as happening on side (i, i + 1). The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.

Time: $\mathcal{O}(\log n)$

```
"Point.h" d41d8c, 39 lines

#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i+1, i) >= 0 && cmp(i, i-1+n) < 0
template <class P> int extrVertex(vector<P>& poly, P dir) {
    int n = sz(poly), lo = 0, hi = n;
    if (extr(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (extr(m)) return m;
        int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
        (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
    }
    return lo;
}

#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
    int endA = extrVertex(poly, (a - b).perp());
    int endB = extrVertex(poly, (b - a).perp());
    if (cmpL(endA) < 0 || cmpL(endB) > 0)
        return {-1, -1};
    array<int, 2> res;
    rep(i,0,2) {
        int lo = endB, hi = endA, n = sz(poly);
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
            (cmpL(m) == cmpL(endB) ? lo : hi) = m;
        }
        res[i] = (lo + !cmpL(hi)) % n;
        swap(endA, endB);
    }
    if (res[0] == res[1]) return {res[0], -1};
    if (!cmpL(res[0]) && !cmpL(res[1]))
        switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
            case 0: return {res[0], res[0]};
            case 2: return {res[1], res[1]};
        }
    return res;
}
```

8.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.

Time: $\mathcal{O}(n \log n)$

```
"Point.h" d41d8c, 17 lines

typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
    assert(sz(v) > 1);
    set<P> S;
    sort(all(v), [](P a, P b) { return a.y < b.y; });
    pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
    int j = 0;
    for (P p : v) {
        P d{1 + (ll)sqrt(ret.first), 0};
```

LineHullIntersection ClosestPair kdTree FastDelaunay

```
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo)
        ret = min(ret, ((*lo - p).dist2(), {*lo, p}));
    S.insert(p);
}
return ret.second;
}
```

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

```
"Point.h" d41d8c, 63 lines

typedef long long T;
typedef Point<T> P;
const T INF = numeric_limits<T>::max();
```

```
bool on_x(const P& a, const P& b) { return a.x < b.x; }
bool on_y(const P& a, const P& b) { return a.y < b.y; }
```

```
struct Node {
    P pt; // if this is a leaf, the single point in it
    T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
    Node *first = 0, *second = 0;

    T distance(const P& p) { // min squared distance to a point
        T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
        T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
        return (P(x,y) - p).dist2();
    }
}
```

```
Node(vector<P>&& vp) : pt(vp[0]) {
    for (P p : vp) {
        x0 = min(x0, p.x); x1 = max(x1, p.x);
        y0 = min(y0, p.y); y1 = max(y1, p.y);
    }
    if (vp.size() > 1) {
        // split on x if width >= height (not ideal...)
        sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
        // divide by taking half the array for each child (not
        // best performance with many duplicates in the middle)
        int half = sz(vp)/2;
        first = new Node({vp.begin(), vp.begin() + half});
        second = new Node({vp.begin() + half, vp.end()});
    }
}
```

```
struct KDTree {
    Node* root;
    KDTree(const vector<P>& vp) : root(new Node({all(vp)})) {}
}
```

```
pair<T, P> search(Node *node, const P& p) {
    if (!node->first) {
        // uncomment if we should not find the point itself:
        // if (p == node->pt) return {INF, P()};
        return make_pair((p - node->pt).dist2(), node->pt);
    }
```

```
Node *f = node->first, *s = node->second;
T bfirst = f->distance(p), bsec = s->distance(p);
if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);
```

```
// search closest side first, other side if needed
auto best = search(f, p);
if (bsec < best.first)
    best = min(best, search(s, p));
return best;
}
```

```
// find nearest point to a point, and its squared distance
// (requires an arbitrary operator< for Point)
pair<T, P> nearest(const P& p) {
    return search(root, p);
}
};
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ...}, all counter-clockwise.

Time: $\mathcal{O}(n \log n)$

```
"Point.h" d41d8c, 88 lines

typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t lll; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX, LLONG_MAX); // not equal to any other point
```

```
struct Quad {
    Q rot, o; P p = arb; bool mark;
    P& F() { return r()->p; }
    Q& r() { return rot->rot; }
    Q prev() { return rot->o->rot; }
    Q next() { return r()->prev(); }
} *H;
```

```
bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
    lll p2 = p.dist2(), A = a.dist2()-p2,
        B = b.dist2()-p2, C = c.dist2()-p2;
    return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
}
```

```
Q makeEdge(P orig, P dest) {
    Q r = H ? H : new Quad{new Quad{new Quad{new Quad{}}}};
    H = r->o; r->r()->r() = r;
    rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
    r->p = orig; r->F() = dest;
    return r;
}
```

```
void splice(Q a, Q b) {
    swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}
```

```
Q connect(Q a, Q b) {
    Q q = makeEdge(a->F(), b->p);
    splice(q, a->next());
    splice(q->r(), b);
    return q;
}
```

```
pair<Q,Q> rec(const vector<P>& s) {
    if (sz(s) <= 3) {
        Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
        if (sz(s) == 2) return { a, a->r() };
        splice(a->r(), b);
        auto side = s[0].cross(s[1], s[2]);
        Q c = side ? connect(b, a) : 0;
        return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
    }
```

```
#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
Q A, B, ra, rb;
int half = sz(s) / 2;
tie(ra, A) = rec({all(s) - half});
tie(B, rb) = rec({sz(s) - half + all(s)});
while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
        (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
Q base = connect(B->r(), A);
```

```
if (A->p == ra->p) ra = base->r();
if (B->p == rb->p) rb = base;

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
while (circ(e->dir->F(), H(base), e->F())) { \
    Q t = e->dir; \
    splice(e, e->prev()); \
    splice(e->r(), e->r()->prev()); \
    e->o = H; H = e; e = t; \
}
for (;;) {
    DEL(LC, base->r(), o); DEL(RC, base, prev());
    if (!valid(LC) && !valid(RC)) break;
    if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
        base = connect(RC, base->r());
    else
        base = connect(base->r(), LC->r());
}
return { ra, rb };
}

vector<P> triangulate(vector<P> pts) {
    sort(all(pts)); assert(unique(all(pts)) == pts.end());
    if (sz(pts) < 2) return {};
    Q e = rec(pts).first;
    vector<Q> q = {e};
    int qi = 0;
    while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
q.push_back(c->r()); c = c->next(); } while (c != e); }
    ADD; pts.clear();
    while (qi < sz(q)) if (!(e = q[qi++])>mark) ADD;
    return pts;
}
```

halfplane.cpp

Description: halfplane intersection do koosaga

d41d8c, 60 lines

```
struct hpi {
    const double eps = 1e-8;
    typedef pair<long double, long double> pi;
    bool z(long double x) const { return fabs(x) < eps; }
    long double ccw(pi a, pi b, pi c) {
        long double dx1 = b.first - a.first;
        long double dy1 = b.second - a.second;
        long double dx2 = c.first - a.first;
        long double dy2 = c.second - a.second;
        return dx1 * dy2 - dy1 * dx2;
    }
    struct line {
        long double a, b, c;
        pi slope() { return pi(a, b); }
    };
    pi cross(line a, line b) {
        long double det = a.a * b.b - b.a * a.b;
        return pi((a.c * b.b - a.b * b.c) / det, (a.a * b.c - a.c * b.a) / det);
    }
    bool bad(line a, line b, line c) {
        if (ccw(pi(0, 0), a.slope(), b.slope()) <= 0)
            return false;
        pi crs = cross(a, b);
        return crs.first * c.a + crs.second * c.b >= c.c;
    }
    bool solve(vector<line> v, vector<pi> &solution) { // ax + by
        <= c;
        sort(v.begin(), v.end(), [&](line &fuck, line &l) {
            auto [a, b, c] = make_tuple(fuck.a, fuck.b, fuck.c);
            bool flag1 = pi(a, b) > pi(0, 0);
```

halfplane PolyhedronVolume Point3D 3dHull

```
bool flag2 = pi(1.a, 1.b) > pi(0, 0);
if (flag1 != flag2)
    return flag1 > flag2;
long double t = ccw(pi(0, 0), pi(a, b), pi(1.a, 1.b));
return z(t) ? c * hypot(1.a, 1.b) < 1.c * hypot(a, b) : t
    > 0;
});
deque<line> dq;
for (auto &i : v) {
    if (!dq.empty() && z(ccw(pi(0, 0), dq.back().slope(), i.
        slope()))
        continue;
    while (dq.size() >= 2 && bad(dq[dq.size() - 2], dq.back()
        , i))
        dq.pop_back();
    while (dq.size() >= 2 && bad(i, dq[0], dq[1]))
        dq.pop_front();
    dq.push_back(i);
}
while (dq.size() > 2 && bad(dq[dq.size() - 2], dq.back(),
    dq[0]))
    dq.pop_back();
while (dq.size() > 2 && bad(dq.back(), dq[0], dq[1]))
    dq.pop_front();
vector<pi> tmp;
for (int i = 0; i < dq.size(); i++) {
    line cur = dq[i], nxt = dq[(i + 1) % dq.size()];
    if (ccw(pi(0, 0), cur.slope(), nxt.slope()) <= eps)
        return false;
    tmp.push_back(cross(cur, nxt));
}
solution = tmp;
return true;
}
} hpi;
```

8.5 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

d41d8c, 6 lines

```
template<class V, class L>
double signedPolyVolume(const V& p, const L& trilst) {
    double v = 0;
    for (auto i : trilst) v += p[i.a].cross(p[i.b]).dot(p[i.c]);
    return v / 6;
}
```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long.

d41d8c, 32 lines

```
template<class T> struct Point3D {
    typedef Point3D P;
    typedef const P& R;
    T x, y, z;
    explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z) {}
    bool operator<(R p) const {
        return tie(x, y, z) < tie(p.x, p.y, p.z); }
    bool operator==(R p) const {
        return tie(x, y, z) == tie(p.x, p.y, p.z); }
    P operator+(R p) const { return P(x+p.x, y+p.y, z+p.z); }
    P operator-(R p) const { return P(x-p.x, y-p.y, z-p.z); }
    P operator*(T d) const { return P(x*d, y*d, z*d); }
    P operator/(T d) const { return P(x/d, y/d, z/d); }
    T dot(R p) const { return x*p.x + y*p.y + z*p.z; }
    P cross(R p) const {
        return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
    }
}
```

```
T dist2() const { return x*x + y*y + z*z; }
double dist() const { return sqrt((double)dist2()); }
//Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
double phi() const { return atan2(y, x); }
//Zenith angle (latitude) to the z-axis in interval [0, pi]
double theta() const { return atan2(sqrt(x*x+y*y),z); }
P unit() const { return *this/(T)dist(); } //makes dist()=1
//returns unit vector normal to *this and p
P normal(P p) const { return cross(p).unit(); }
//returns point rotated 'angle' radians ccw around axis
P rotate(double angle, P axis) const {
    double s = sin(angle), c = cos(angle); P u = axis.unit();
    return u.dot(u)*(1-c) + (*this)*c - cross(u)*s;
}
};
```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.

Time: $\mathcal{O}(n^2)$

"Point3D.h" d41d8c, 49 lines

```
typedef Point3D<double> P3;

struct PR {
    void ins(int x) { (a == -1 ? a : b) = x; }
    void rem(int x) { (a == x ? a : b) = -1; }
    int cnt() { return (a != -1) + (b != -1); }
    int a, b;
};

struct F { P3 q; int a, b, c; };

vector<F> hull3d(const vector<P3>& A) {
    assert(sz(A) >= 4);
    vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
#define E(x,y) E[f.x][f.y]
    vector<F> FS;
    auto mf = [&](int i, int j, int k, int l) {
        P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
        if (q.dot(A[l]) > q.dot(A[i]))
            q = q * -1;
        F f{q, i, j, k};
        E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
        FS.push_back(f);
    };
    rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
        mf(i, j, k, 6 - i - j - k);

    rep(i,4,sz(A)) {
        rep(j,0,sz(FS)) {
            F f = FS[j];
            if(f.q.dot(A[i]) > f.q.dot(A[f.a])) {
                E(a,b).rem(f.c);
                E(a,c).rem(f.b);
                E(b,c).rem(f.a);
                swap(FS[j--], FS.back());
                FS.pop_back();
            }
        }
        int nw = sz(FS);
        rep(j,0,nw) {
            F f = FS[j];
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
            C(a, b, c); C(a, c, b); C(b, c, a);
        }
    }
    for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
        A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
```

```
    return FS;
};
```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.

```
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
    double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
    double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
    double dz = cos(t2) - cos(t1);
    double d = sqrt(dx*dx + dy*dy + dz*dz);
    return radius*2*asin(d/2);
}
```

Strings (9)

SuffixArray.cpp

Description: kasai recebe o suffix array e calcula lcp[i], o lcp entre s[sa[i], ..., n - 1] e s[sa[i + 1], ..., n - 1].
Time: $\mathcal{O}(N \log N)$. Kasai in $\mathcal{O}(N)$.

```
vector<int> suffix_array(string s) {
    s += "$";
    int n = s.size(), N = max(n, 260);
    vector<int> sa(n), ra(n);
    for(int i = 0; i < n; i++) sa[i] = i, ra[i] = s[i];

    for(int k = 0; k < n; k ? k *= 2 : k++) {
        vector<int> nsa(sa), nra(n), cnt(N);

        for(int i = 0; i < n; i++) nsa[i] = (nsa[i]-k+n)%n, cnt[ra[i]]++;
        for(int i = 1; i < N; i++) cnt[i] += cnt[i-1];
        for(int i = n-1; i+1; i--) sa[--cnt[ra[nsa[i]]]] = nsa[i];

        for(int i = 1, r = 0; i < n; i++) nra[sa[i]] = r += ra[sa[i-1]] !=
            ra[sa[i-1]] or ra[(sa[i]+k)%n] != ra[(sa[i-1]+k)%n];
        ra = nra;
        if (ra[sa[n-1]] == n-1) break;
    }
    return vector<int>(sa.begin()+1, sa.end());
}
```

```
vector<int> kasai(string s, vector<int> sa) {
    int n = s.size(), k = 0;
    vector<int> ra(n), lcp(n);
    for (int i = 0; i < n; i++) ra[sa[i]] = i;

    for (int i = 0; i < n; i++, k -= !!k) {
        if (ra[i] == n-1) { k = 0; continue; }
        int j = sa[ra[i]+1];
        while (i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
        lcp[ra[i]] = k;
    }
    return lcp;
}
```

Manacher.cpp

Description: manacher recebe um vetor de T e retorna o vetor com tamanho dos palindromos ret[2*i] = tamanho do maior palindromo centrado em i
ret[2*i+1] = tamanho maior palindromo centrado em i e i+1
Time: all in $\mathcal{O}(N)$.

```
template<typename T> vector<int> manacher(const T& s) {
    int l = 0, r = -1, n = s.size();
    vector<int> dl(n), d2(n);
    for (int i = 0; i < n; i++) {
        int k = i > r ? 1 : min(dl[l+r-i], r-i);
        while (i+k < n && i-k >= 0 && s[i+k] == s[i-k]) k++;
        dl[i] = k--;
        if (i+k > r) l = i-k, r = i+k;
    }
    l = 0, r = -1;
    for (int i = 0; i < n; i++) {
        int k = i > r ? 0 : min(d2[l+r-i+1], r-i+1); k++;
        while (i+k <= n && i-k >= 0 && s[i+k-1] == s[i-k]) k++;
        d2[i] = --k;
        if (i+k-1 > r) l = i-k, r = i+k-1;
    }
    vector<int> ret(2*n-1);
    for (int i = 0; i < n; i++) ret[2*i] = 2*dl[i]-1;
    for (int i = 0; i < n-1; i++) ret[2*i+1] = 2*d2[i+1];
    return ret;
}
```

```
// verifica se a string s[i..j] eh palindromo
template<typename T> struct palindrome {
    vector<int> man;

    palindrome(const T& s) : man(manacher(s)) {}
    bool query(int l, int j) {
        return man[i+j] >= j-i+1;
    }
};
```

```
// tamanho do maior palindromo que termina em cada posicao
template<typename T> vector<int> pal_end(const T& s) {
    vector<int> ret(s.size());
    palindrome<T> p(s);
    ret[0] = 1;
    for (int i = 1; i < s.size(); i++) {
        ret[i] = min(ret[i-1]+2, i+1);
        while (!p.query(i-ret[i]+1, i)) ret[i]--;
    }
    return ret;
}
```

Lyndon.cpp

Description: Duval algorithm to find the Lyndon factorization of a string. Also contains a function to find the minimum cyclic shift of a string.
Time: All $\mathcal{O}(n)$

```
vector<string> duval(string const &s) {
    int n = s.size();
    int i = 0;
    vector<string> factorization;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j]) k = i;
            else k++;
            j++;
        }
        while (i <= k) {
            factorization.push_back(s.substr(i, j - k));
            i += j - k;
        }
    }
}
```

```
    }
    return factorization;
}

string min_cyclic_shift(string s) {
    s += s;
    int n = s.size();
    int i = 0, ans = 0;
    while (i < n / 2) {
        ans = i;
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j]) k = i;
            else k++;
            j++;
        }
        while (i <= k) i += j - k;
    }
    return s.substr(ans, n / 2);
}
```

EertreeE.cpp

Description: palindromic tree, palindromic factorization
Time: $\mathcal{O}(N)$ to build, can use map instead of array on 'to' for memory optimization. palindromic factorization computa o numero de formas de particionar cada prefixo da string em strings palindromicas

```
const int N = 2e5 + 15;
const int ALF = 26;

struct eertree {
    int str[N], len[N], lnk[N], cnt[N], first[N], node_cnt, it,
        last;
    ll palindrome_substring_sum;
    const char norm = 'a';
    array<int, ALF> to[N];

    inline int get(char c) { return c - norm; }

    void init(int n) {
        memset(str, 0, sizeof(int) * (n + 1));
        memset(len, 0, sizeof(int) * (n + 1));
        memset(lnk, 0, sizeof(int) * (n + 1));
        memset(cnt, 0, sizeof(int) * (n + 1));
        for (int i = 0; i <= n; i++)
            for (int j = 0; j < ALF; j++) to[i][j] = 0;
        node_cnt = 2, it = 1, last = 0, str[0] = -1;
        len[0] = 0, len[1] = -1, lnk[0] = 1, lnk[1] = 1;
    }

    void set_string(const string &s) {
        int n = (int)s.size();
        init(n);
        for (int i = 0; i < n; i++) insert(s[i]);
        build_cnt();
    }

    int insert(char ch) {
        int c = get(ch);
        str[it] = c;
        while (str[it - 1 - len[last]] != c) last = lnk[last];
        if (!to[last][c]) {
            int prev = lnk[last];
            while (str[it - 1 - len[prev]] != c) prev = lnk[prev];
            lnk[node_cnt] = to[prev][c];
            len[node_cnt] = len[last] + 2;
            to[last][c] = node_cnt++;
        }
    }
}
```

```

        last = to[last][c];
        first[last] = it;
        cnt[last]++;
        it++;
        return last;
    }

    void build_cnt() {
        ll ans = 0;
        for (int i = it; i > 1; i--) {
            ans += cnt[i];
            cnt[lnk[i]] += cnt[i];
        }
        palindrome_substring_sum = ans;
    }

    inline ll number_of_palindromes() { return
        palindrome_substring_sum; }
    inline int number_of_distinct_palindromes() { return
        node_cnt - 2; }

} et;

ll factorization(string s) {
    int n = s.size(), sz = 2;
    eertree PT(n);
    vector<int> diff(n+2), slink(n+2), sans(n+2), dp(n+1);
    dp[0] = 1;
    for (int i = 1; i <= n; i++) {
        PT.insert(s[i-1]);
        if (PT.size()+2 > sz) {
            diff[sz] = PT.len[sz] - PT.len[PT.lnk[sz]];
            if (diff[sz] == diff[PT.lnk[sz]])
                slink[sz] = slink[PT.lnk[sz]];
            else slink[sz] = PT.lnk[sz];
            sz++;
        }
        for (int v = PT.last; PT.len[v] > 0; v = slink[v]) {
            sans[v] = dp[i - (PT.len[slink[v]] + diff[v])];
            if (diff[v] == diff[PT.lnk[v]])
                sans[v] = (sans[v] + sans[PT.lnk[v]]) % MOD;
            dp[i] = (dp[i] + sans[v]) % MOD;
        }
    }
    return dp[n];
}

```

Various (10)

simulatedAnnealing.cpp

d41d8c, 46 lines

```

bool P(double E,double E_next,double T,mt19937 rng){
    double prob = exp(-(E_next-E)/T);
    if(prob > 1) return true;
    else{
        bernoulli_distribution d(prob);
        return d(rng);
    }
}

class state {
public:
    state() {
        // Generate the initial state
    }
    state next() {
        state s_next;
        // Modify s_next to a random neighboring state
        return s_next;
    }
}

```

```

double E() {
    // Implement the energy function here
};

pair<double, state> simAnneal() {
    state s = state();
    state best = s;
    double T = 10000; // Initial temperature
    double u = 0.995; // decay rate
    double E = s.E();
    double E_next;
    double E_best = E;
    mt19937 rng(chrono::steady_clock::now().time_since_epoch().
        count());
    while (T > 1) {
        state next = s.next();
        E_next = next.E();
        if (P(E, E_next, T, rng)) {
            s = next;
            if (E_next < E_best) {
                best = s;
                E_best = E_next;
            }
            E = E_next;
        }
        T *= u;
    }
    return {E_best, best};
}

```

10.1 Optimization tricks

10.1.1 Pragmas

- `#pragma GCC optimize ("Ofast")` will make GCC auto-vectorize loops and optimizes floating points better.
- `#pragma GCC target ("avx2")` can double performance of vectorized code, but causes crashes on old machines.

FastMod.h

Description: Compute $a\%b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to $a \pmod b$ in the range $[0, 2b)$.

d41d8c, 8 lines

```

typedef unsigned long long ull;
struct FastMod {
    ull b, m;
    FastMod(ull b) : b(b), m(~1ULL / b) {}
    ull reduce(ull a) { // a % b + (0 or b)
        return a - (ull)((__uint128_t(m) * a) >> 64) * b;
    }
};

```

FastInput.h

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.

Usage: `./a.out < input.txt`

Time: About 5x as fast as `cin/scanf`.

d41d8c, 17 lines

```

inline char gc() { // like getchar()
    static char buf[1 << 16];
    static size_t bc, be;
    if (bc >= be) {
        buf[0] = 0, bc = 0;
        be = fread(buf, 1, sizeof(buf), stdin);
    }
    return buf[bc++]; // returns 0 on EOF
}

```

```

int readInt() {
    int a, c;
    while ((a = gc()) < 40);
    if (a == '-') return -readInt();
    while ((c = gc()) >= 48) a = a * 10 + c - 480;
    return a - 48;
}

```

10.2 DP Optimizations

10.2.1 Quadrangle Inequality (QI)

For $a \leq b \leq c \leq d$, $C(a, c) + C(b, d) \leq C(a, d) + C(b, c)$.

10.2.2 Knuth Optimization

Applies to $dp[i][j] = cost[i][j] + \min_{i \leq k < j} (dp[i][k] + dp[k+1][j])$.

Requires $opt[i][j-1] \leq opt[i][j] \leq opt[i+1][j]$. Sufficient: $cost$ satisfies QI and $cost[b][c] \leq cost[a][d]$ ($a \leq b \leq c \leq d$). Search $k \in [opt[i][j-1], opt[i+1][j]]$. Time $O(N^2)$.

10.2.3 Divide & Conquer DP

$dp(i, j) = \min_{0 \leq k \leq j} (dp(i-1, k-1) + C(k, j))$. Requires $opt(i, j) \leq opt(i, j+1)$. Sufficient: C satisfies QI. Time $O(mn \log n)$.

10.2.4 1D1D Optimization

$dp[i] = \min_{j < i} (dp[j] + cost(j, i))$. Requires $par[i] \leq par[i+1]$. Sufficient: $cost$ is QI. Time $O(n \log n)$.